Nombres: Equipo 11 Christopher Cortez Garcia Oscar Elias Vera Moreno Christofer Aaron Torres Puga	**Matrículas:** 2159313 2160859 2159639
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Módulo Netacad: 6	**Actividad:** Reporte de Solución de Software Basada en Lógica de Negocio — FitZone (Caso 11)
Fecha: 06/11/2026	
Bibliografía (Formato APA) Facultad de Ingeniería Mecánica y Eléctrica. (2026). Producto Integrador de Aprendizaje (PIA): Implementación de una solución de software basada en lógica de negocio — Caso 11 FitZone. Universidad Autónoma de Nuevo León. Evans, E. (2003). Domain-Driven Design. Addison-Wesley.	

Resumen ejecutivo

El presente reporte documenta el desarrollo del Producto Integrador de Aprendizaje (PIA) del caso FitZone, asignado al Equipo 11. El objetivo fue convertir los artefactos de análisis elaborados en las unidades anteriores en una solución de software organizada por capas, con reglas de negocio modeladas en el dominio, servicios de aplicación, persistencia, API REST, interfaz web, pruebas automatizadas y despliegue.

El trabajo se organizó en cuatro fases que corresponden a los bloques de evaluación del PIA: análisis del negocio, arquitectura, implementación y calidad/producción/defensa. La documentación conserva la trazabilidad entre las necesidades del gimnasio, las historias de usuario, los procesos BPMN, el modelo de dominio, los endpoints y las pruebas.

En la validación local final con Docker Compose se levantaron PostgreSQL, la API y el worker como servicios separados. La ejecución del contenedor de pruebas reportó 34 pruebas aprobadas y 91.43% de cobertura, por encima del mínimo solicitado de 80%. El pipeline de GitHub Actions también cuenta con una ejecución exitosa que construye los servicios, verifica salud y ejecuta las pruebas.

El sistema cubre los flujos que están implementados y comprobados; no se presenta como terminada la integración con dispositivos físicos, pasarelas reales de pago ni el envío externo de correos o SMS. Estas limitaciones se detallan para distinguir las capacidades demostradas de las extensiones futuras.

Propósito y objetivos

Propósito. Diseñar e implementar una solución de software para FitZone que permita representar y controlar procesos centrales del gimnasio —membresías, socios, clases, reservas, asistencia, acceso y reportes— mediante reglas explícitas, una arquitectura mantenible y evidencia verificable.

Objetivo general

Integrar los requerimientos y procesos del caso FitZone en una aplicación funcional basada en lógica de negocio, aplicando DDD, Specification, State Machine, API REST, persistencia, pruebas automatizadas, CI/CD y contenedorización.

Objetivos específicos

- Refinar el BRD y relacionar sus reglas con historias de usuario, escenarios Gherkin, BPMN y glosario.
- Modelar el dominio con entidades, agregados, objetos de valor, eventos y servicios coherentes con el caso.
- Implementar casos de uso y exponerlos mediante una API REST documentada con OpenAPI/Swagger.
- Integrar persistencia, interfaz web y tareas programadas donde corresponde.
- Verificar el comportamiento con pruebas unitarias y de integración, alcanzar al menos 80% de cobertura y automatizar validaciones.
- Preparar una ejecución reproducible con Docker Compose, variables de entorno y despliegue demostrable.

Alcance y organización del trabajo

La guía oficial del PIA solicita integrar los productos de las Unidades 2 y 3; implementar DDD, Specification, State Machine y API REST; organizar el repositorio; comprobar pruebas con cobertura mínima de 80%; mantener CI/CD; usar Docker Compose para base de datos, aplicación y pruebas; separar configuración con variables de entorno; y disponer de un despliegue con Swagger/OpenAPI para la demostración.

Fase del proyecto	Propósito	Producto principal
1. Análisis del negocio	Entender el problema, las reglas y los procesos del gimnasio.	BRD, historias Gherkin, BPMN y glosario.
2. Arquitectura	Traducir el análisis a un modelo de software mantenible.	DDD, UML, capas, patrones, SOLID y ADRs.
3. Implementación	Construir los componentes y flujos de negocio.	Dominio, aplicación, infraestructura, API, worker e interfaz.
4. Calidad y producción	Comprobar, automatizar y preparar la demostración.	Pruebas, cobertura, CI/CD, Docker Compose y despliegue.

1. Fase de análisis del negocio

1.1 Contexto y problemática

FitZone ofrece membresías de distintas vigencias, clases grupales con cupo limitado y servicios de entrenamiento. El análisis del caso identificó dificultades operativas: espacios que quedan vacíos por inasistencias, listas de espera administradas manualmente, validaciones de acceso que pueden no reflejar la vigencia real de una membresía y falta de información consolidada sobre ocupación y asistencia.

El análisis parte de los documentos elaborados por el equipo en las unidades previas: BRD, historias de usuario, diagrama BPMN y glosario de dominio. Estos artefactos se integraron como base de las decisiones técnicas; no se trataron como documentación aislada.

Levantamiento de Requerimientos (Entrevista con Stakeholder)

#	Pregunta formulada al Stakeholder	Respuesta / Regla de Negocio Confirmada
P1	¿Qué canales puede usar el socio para reservar una clase?	App móvil/web responsiva o presencialmente en recepción (también vía telefónica asistida por recepción).
P2	¿Existe un tiempo límite de anticipación para reservar?	Hasta con 7 días de anticipación máximo para evitar saturar el calendario futuro.
P3	¿El instructor puede cancelar una clase por sí mismo?	Sí, pero debe registrarlo y notificar al administrador con al menos 2 horas de anticipación.
P4	¿Cuántas clases simultáneas puede reservar un socio según su plan?	El Plan Básico permite máximo 2 reservas activas simultáneas; el Plan Premium/Preferente no tiene límite.
P5	¿Hasta cuántas horas antes se puede cancelar una reserva sin penalización?	Hasta 3 horas antes del inicio de la clase (Cut-off Time) sin penalización. Cancelar con menos de 3 horas cuenta como cancelación tardía / falta.

P6	¿Cuántos no-shows generan sanción o bloqueo?	3 no-shows acumulados en el mes generan un bloqueo automático de 7 días para realizar nuevas reservas de clases (el acceso físico general al gimnasio permanece permitido si la membresía está vigente).
P7	¿Cuánto tiempo tiene un socio en lista de espera para confirmar un lugar liberado?	30 minutos desde la notificación; si no confirma en esa ventana, el lugar pasa automáticamente al siguiente en la fila.
P8	¿Cómo se ordena la lista de espera?	Orden estricto de llegada (FIFO — First-In, First-Out), sin privilegios comerciales por tipo de plan.
P9	¿Qué ocurre con las reservas futuras si vence la membresía del socio?	Se cancelan automáticamente al vencer la membresía y se liberan los cupos hacia la lista de espera.
P10	¿Qué pasa cuando un instructor cancela una clase?	Se notifica a los socios inscritos y se ofrece reagendamiento sin costo; no existen reembolsos monetarios unitarios porque las clases son parte de la suscripción.
P11	¿Se permite asistir a una clase no incluida en el plan del socio?	Sí, mediante un pago único de acceso extraordinario (Drop-in), sin alterar el plan base de su membresía.
P12	¿Cuáles son las reglas comerciales de contratación de planes y beneficios VIP?	Toda solicitud de operación/membresía inicia en estado SOLICITADA, requiere socio activo, al menos un plan/recurso habilitado con stock disponible y autorización. Los socios Preferentes (VIP) con importe total >= $1,500.00 MXN y sin promoción activa reciben un beneficio del 10% de descuento.
P13	¿Cuál es la duración oficial de cada tipo de clase y cómo se determina la disponibilidad del instructor?	Pendiente de validar con el stakeholder; no se asigna una regla sin confirmación.
P14	¿Qué canales oficiales se utilizarán para notificar promociones FIFO, cancelaciones y alertas de vencimiento?	Pendiente de validar con el stakeholder; el sistema actual conserva avisos en outbox, sin entrega externa.
P15	¿Qué datos y evidencias debe conservar recepción al registrar reservas u operaciones para un socio?	Pendiente de validar con el stakeholder. Implementación parcial: existe una bitácora persistente de acciones administrativas y validaciones de acceso con actor, rol, entidad, fecha y resultado; aún debe confirmarse si esos datos cubren todos los registros requeridos por recepción.

1.2 Levantamiento y reglas del negocio

El BRD registra necesidades, actores, restricciones y reglas que condicionan la operación. Entre las reglas principales documentadas se encuentran:

- Las reservas pueden realizarse por los canales definidos para el gimnasio y con una anticipación máxima de siete días.
- El plan Básico limita las reservas activas simultáneas; los planes Premium/Preferente manejan una política distinta.
- La cancelación de una reserva sin penalización depende de la anticipación respecto al inicio de la clase.
- La acumulación de inasistencias puede provocar una suspensión temporal de nuevas reservas, sin equivaler necesariamente a bloquear el acceso físico al gimnasio.
- La lista de espera se atiende por orden de llegada (FIFO) y el socio promovido dispone de una ventana para confirmar el lugar liberado.
- El vencimiento de una membresía afecta las reservas futuras activas; también se contempla la modalidad Drop-in para determinados casos.

Las reglas concretas se deben consultar en el BRD del repositorio, que conserva el detalle del levantamiento. La implementación no sustituye reglas faltantes con fechas o integraciones supuestas.

1.3 Historias de usuario y criterios Gherkin

El conjunto refinado contiene 20 historias de usuario identificadas como US-001 a US-020, superando el mínimo de 15 establecido por la guía. Cada historia expresa una necesidad desde la perspectiva de un actor y contiene escenarios de aceptación en formato Given/When/Then (Dado/Cuando/Entonces).

Los escenarios Gherkin se utilizan como puente entre el análisis y la verificación: ayudan a determinar qué comportamiento debe ofrecer el sistema y qué condiciones deben comprobarse mediante servicios, endpoints y pruebas.

Matriz de Necesidades de Negocio (N1 – N6)

ID	Necesidad de Negocio	Descripción	Criterio de Éxito Medible
N1	Validación Automática de Acceso	Verificar automáticamente la vigencia y estado de la membresía del socio en recepción/torniquete.	100% de los accesos físicos validados sin intervención manual, indicando motivo exacto en rechazos.
N2	Gestión de Membresías y Reservas	Permitir contratar planes deportivos, evaluar beneficios VIP y reservar/cancelar clases respetando límites por plan y ventana de cut-off (3h).	0% de sobreventa de capacidad máxima o contrataciones de planes deshabilitados.

N3	Automatización de Lista de Espera	Administrar cola FIFO cuando una clase agota su cupo y reasignar automáticamente lugares liberados con ventana de 30 min.	100% de lugares cancelados a tiempo asignados en orden estricto de llegada (FIFO).
N4	Control de Asistencia y Sanciones	Registrar asistencia/inasistencia y aplicar bloqueo de 7 días para reservas al acumular 3 no-shows en el mes.	Reducción de no-shows >= 20% en 3 meses y aplicación determinista de sanciones.
N5	Analítica y Reportes Operativos	Proveer métricas en tiempo real de porcentaje de ocupación por clase, tasa de no-shows y alertas de vencimiento.	Disponibilidad inmediata vía API REST de indicadores de demanda y morosidad/inasistencia.
N6	Continuidad del Servicio	Permitir asignar un instructor suplente cuando el titular no asiste o cancelar con >= 2h de aviso reagendando a los socios.	100% de eventos de contingencia notificados a los socios inscritos.

Catálogo de 20 Historias de Usuario (User Stories) con Criterios Gherkin

ID	Rol / Historia de Usuario	Escenario Gherkin (Given / When / Then)	Necesidad / Regla
US-001	**Como** socio, **quiero** reservar un lugar en una clase, **para** asegurar mi participación.	**Given** un socio con membresía vigente, sin bloqueo disciplinario y una clase incluida en su plan con cupo disponible (`cupo > 0`) **When** el socio solicita reservar un lugar en esa clase **Then** el sistema confirma la reserva (`CONFIRMADA`), descuenta 1 del cupo disponible y emite el evento `ReservaConfirmada`.	N2 · CU-01
US-002	**Como** socio, **quiero** consultar la disponibilidad de una clase antes de reservar, **para** decidir si me anoto o busco otro horario.	**Given** una clase programada con horario, instructor y capacidad máxima definidos **When** el socio consulta el detalle de la clase **Then** el sistema muestra el cupo disponible actual, el total en lista de espera y si está incluida en su plan.	N2 · CQRS Query
US-003	**Como** socio, **quiero** anotarme en la lista de espera cuando una clase está llena, **para** tener oportunidad de asistir si se libera un lugar.	**Given** una clase sin cupo disponible (`cupo == 0`) y un socio elegible **When** el socio solicita reservar en esa clase **Then** el sistema lo registra en la `ListaDeEspera` en orden FIFO indicando su posición exacta.	N3 · CU-02
US-004	**Como** socio, **quiero** cancelar mi reserva con al menos 3 horas de anticipación, **para** liberar mi lugar a otro socio sin penalización.	**Given** una reserva confirmada con `>= 3 horas` de anticipación al inicio de la clase **When** el socio solicita cancelar su reserva **Then** el sistema cambia la reserva a `CANCELADA` sin aplicar sanción, libera el cupo y emite `LugarLiberadoEnClase`.	N2, N3 · CU-03

US-005	**Como** socio en espera, **quiero** ser promovido cuando se libere un lugar, **para** confirmar mi asistencia dentro del tiempo límite de 30 minutos.	**Given** un socio en la primera posición de la lista de espera (FIFO) y un cupo recién liberado **When** el sistema procesa la liberación del lugar **Then** promueve automáticamente al primer socio en espera, le notifica y le otorga 30 minutos para confirmar antes de pasar al siguiente.	N3 · CU-02, CU-03
US-006	**Como** recepción, **quiero** verificar la vigencia de la membresía de un socio al ingresar, **para** autorizar o negar el acceso físico correctamente.	**Given** un socio que se identifica en el torniquete de recepción **When** el sistema consulta el estado de su membresía **Then** autoriza el acceso si está `ACTIVA` y vigente, incluso si el socio tiene bloqueo temporal de reservas por *no-shows*.	N1 · CU-04
US-007	**Como** recepción, **quiero** ver el motivo explícito cuando el acceso de un socio es rechazado, **para** explicarle su situación con claridad.	**Given** un socio con membresía `VENCIDA`, `SUSPENDIDA` o inexistente **When** intenta ingresar por recepción **Then** el sistema deniega el paso, emite `AccesoFisicoDenegado` y devuelve el motivo específico (`"Membresía Vencida"` o `"Membresía Suspendida"`).	N1 · CU-04
US-008	**Como** recepción, **quiero** registrar manualmente una reserva o solicitud de membresía para un socio, **para** atender solicitudes presenciales.	**Given** un socio activo y planes deportivos habilitados en catálogo **When** recepción registra la operación a nombre del socio **Then** el sistema valida las mismas reglas de dominio (`Specification`) y crea la operación en estado `SOLICITADA`.	N2 · CU-01
US-009	**Como** instructor, **quiero** ver la lista de socios inscritos en mi clase, **para** prepararla y pasar asistencia.	**Given** una clase programada próxima a impartirse **When** el instructor consulta el padrón de la sesión **Then** el sistema devuelve la lista de socios con reserva confirmada y los socios en espera.	N4
US-010	**Como** instructor, **quiero** cancelar una clase con al menos 2 horas de anticipación, **para** notificar a los socios y evitar asistencias en vano.	**Given** una clase programada con socios inscritos y `>= 2 horas` antes de su inicio **When** el instructor cancela la sesión **Then** el sistema marca la clase como cancelada, emite `ClaseCanceladaPorInstructor` y notifica a los inscritos para reagendar sin costo.	N6
US-011	**Como** instructor, **quiero** registrar la asistencia o inasistencia (*no-show*) de cada socio, **para** mantener un historial confiable.	**Given** una clase en curso con socios confirmados **When** el instructor marca asistencia o *no-show* para cada socio **Then** el sistema actualiza el estado de la reserva y el contador mensual de inasistencias del socio.	N4 · CU-05
US-012	**Como** administrador, **quiero** definir la capacidad máxima y	**Given** un plan o clase en el catálogo de FitZone **When** el administrador establece su `capacidad_o_stock` y estado `habilitado`	N2

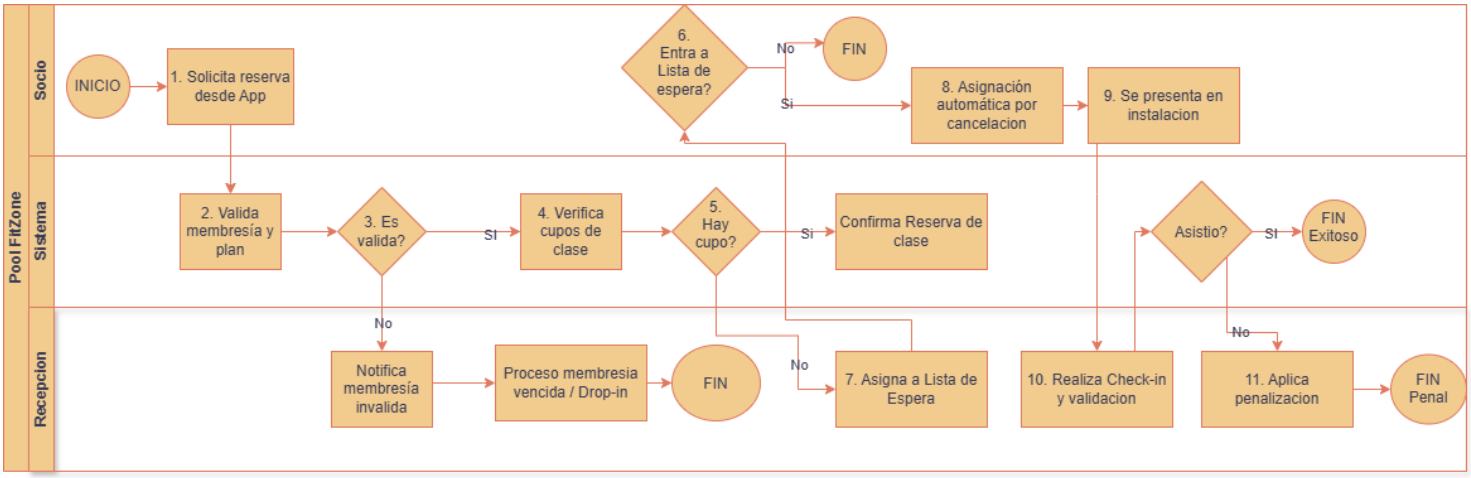
	stock de cada clase o plan, **para** evitar sobrecupo.	**Then** el dominio impide cualquier operación que supere la capacidad o utilice un recurso deshabilitado.	
US-013	**Como** administrador, **quiero** aplicar límites de reservas simultáneas según el plan del socio y beneficios VIP en contrataciones, **para** reflejar las políticas comerciales.	**Given** un socio con Plan Básico (máx. 2 reservas) o un Socio Preferente (VIP) con operación `>= $1,500 MXN` sin promoción activa **When** se evalúa la regla comercial mediante `Specification` **Then** el sistema restringe la 3ra reserva en Plan Básico y otorga 10% de beneficio en contrataciones elegibles.	N2 · Spec Pattern
US-014	**Como** administrador, **quiero** consultar reportes de ocupación por clase y estado de operaciones, **para** identificar demanda real.	**Given** clases programadas y operaciones de membresía registradas **When** el administrador solicita el reporte de ocupación **Then** el sistema calcula y devuelve el porcentaje de ocupación (`reservas / capacidad_maxima * 100`) por clase.	N5
US-015	**Como** administrador, **quiero** consultar la tasa de *no-shows* por socio, **para** auditar las penalizaciones aplicadas.	**Given** el historial de reservas asistidas y no asistidas de los socios **When** el administrador consulta las métricas de inasistencia **Then** el sistema muestra el total de *no-shows*, la tasa porcentual y si el socio se encuentra sancionado.	N4, N5
US-016	**Como** administrador, **quiero** recibir alertas de membresías próximas a vencer, **para** gestionar renovaciones oportunamente.	**Given** membresías activas cuya fecha de fin está dentro de los próximos `N` días **When** se ejecuta la consulta de alertas de vencimiento **Then** el sistema lista los socios cuyas membresías vencerán en dicho umbral.	N5
US-017	**Como** sistema, **quiero** bloquear automáticamente las reservas de un socio que acumula 3 *no-shows* en el mes, **para** aplicar la política disciplinaria sin sesgos.	**Given** un socio con 2 *no-shows* en el mes en curso que incurre en su tercer *no-show* **When** el dominio procesa la tercera inasistencia **Then** bloquea al socio para nuevas reservas durante **7 días**, emite `SocioBloqueadoPorInasistencia` y conserva intacto su acceso físico al gimnasio.	N4 · CU-05
US-018	**Como** sistema, **quiero** cancelar automáticamente las reservas activas de un socio cuando su membresía vence, **para** evitar que ocupe lugares sin contrato vigente.	**Given** un socio con membresía que transita al estado `VENCIDA` y que posee reservas futuras activas **When** se procesa el vencimiento de la membresía **Then** el sistema cancela sus reservas activas, libera los cupos, promueve de la lista de espera y emite `MembresiaVencida`.	N1, N2
US-019	**Como** socio, **quiero** pagar un acceso extraordinario "*drop-in*" a una clase fuera de mi plan, **para** asistir	**Given** una clase con cupo disponible no incluida en el plan base del socio **When** el socio solicita inscripción *drop-in* cubriendo la tarifa unitaria **Then** el `ServicioDeInscripcionExtraordinaria`	N2

	ocasionalmente sin cambiar de contrato.	confirma la reserva con marca `es_drop_in=True` sin alterar su membresía base.	
US-020	**Como** recepción, **quiero** asignar un instructor de respaldo cuando el titular no se presenta, **para** garantizar la continuidad de la sesión.	**Given** una clase próxima a iniciar cuyo instructor titular reporta ausencia imprevista **When** recepción asigna un instructor suplente disponible en ese horario **Then** la clase actualiza su instructor titular sin cancelar las reservas de los socios; si no hay suplente, se cancela la clase notificando a todos.	N6

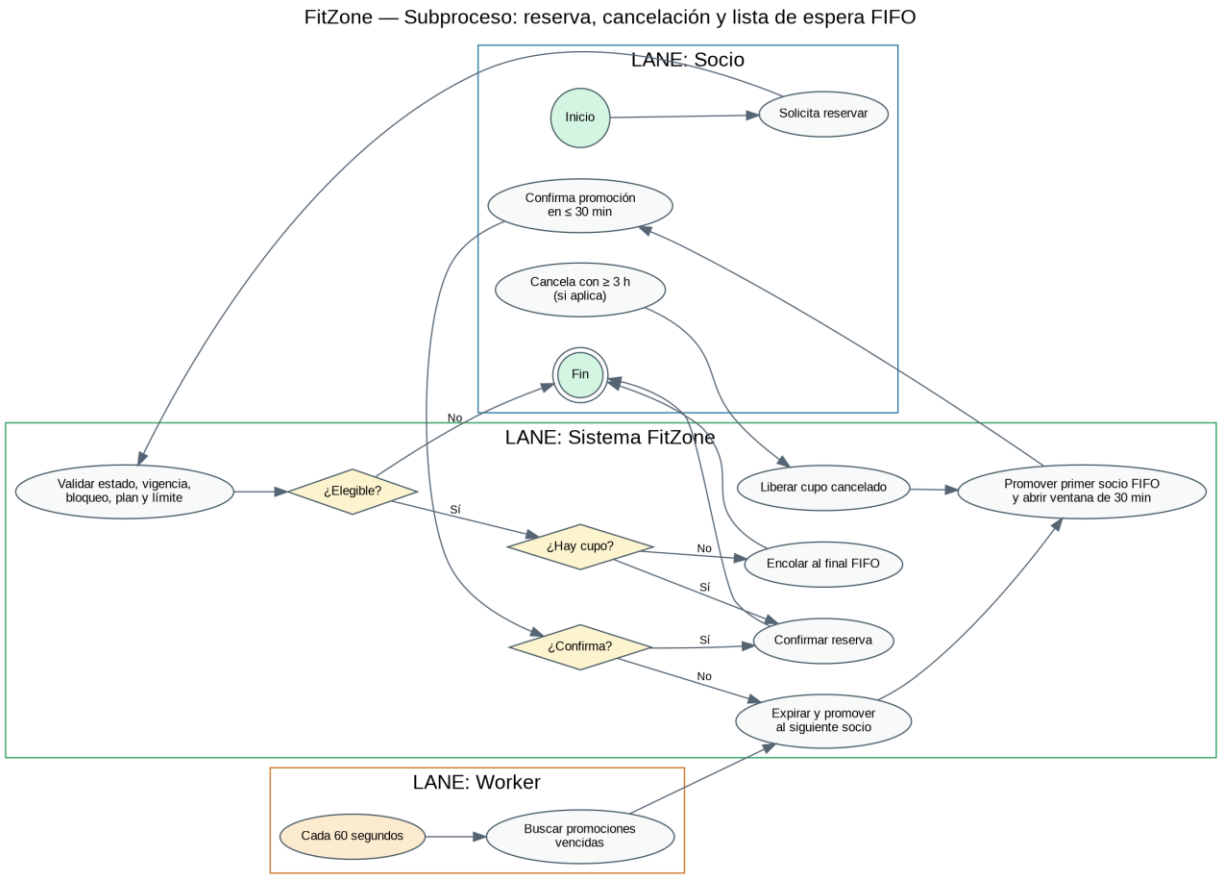
1.4 Procesos BPMN y glosario

Los diagramas BPMN describen los procesos del gimnasio, los participantes y las decisiones que modifican el flujo. Para FitZone son especialmente relevantes los procesos de membresías, reservas FIFO, asistencia/no-show y acceso. El glosario fija el significado de términos como socio, membresía, plan, reserva, lista de espera, no-show, operación y vigencia, evitando que el mismo concepto se interprete de forma diferente en análisis y código.

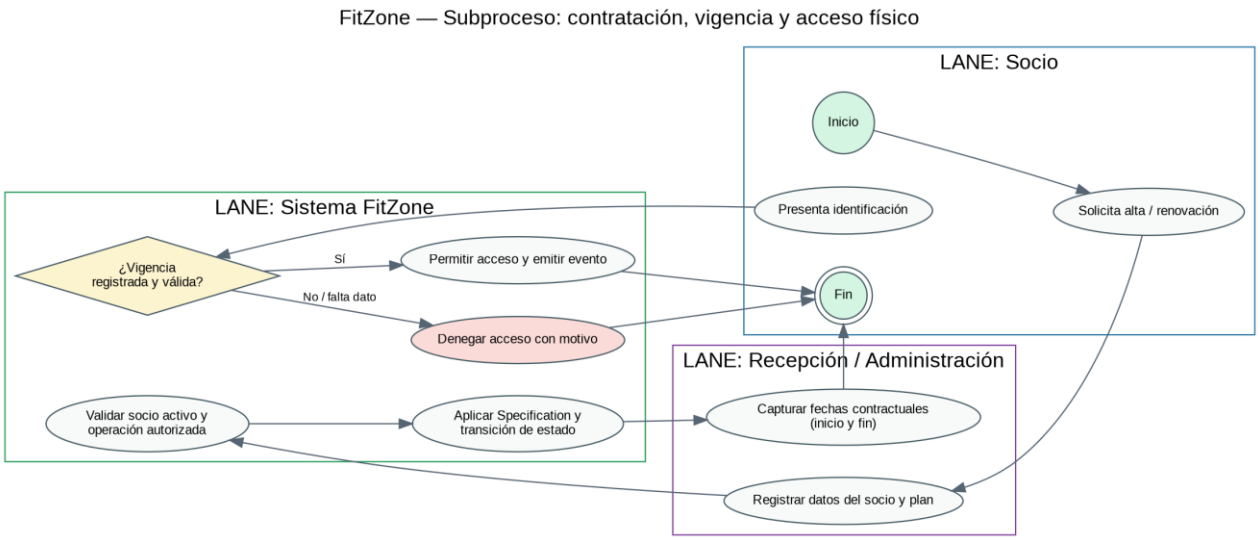
BPMN General



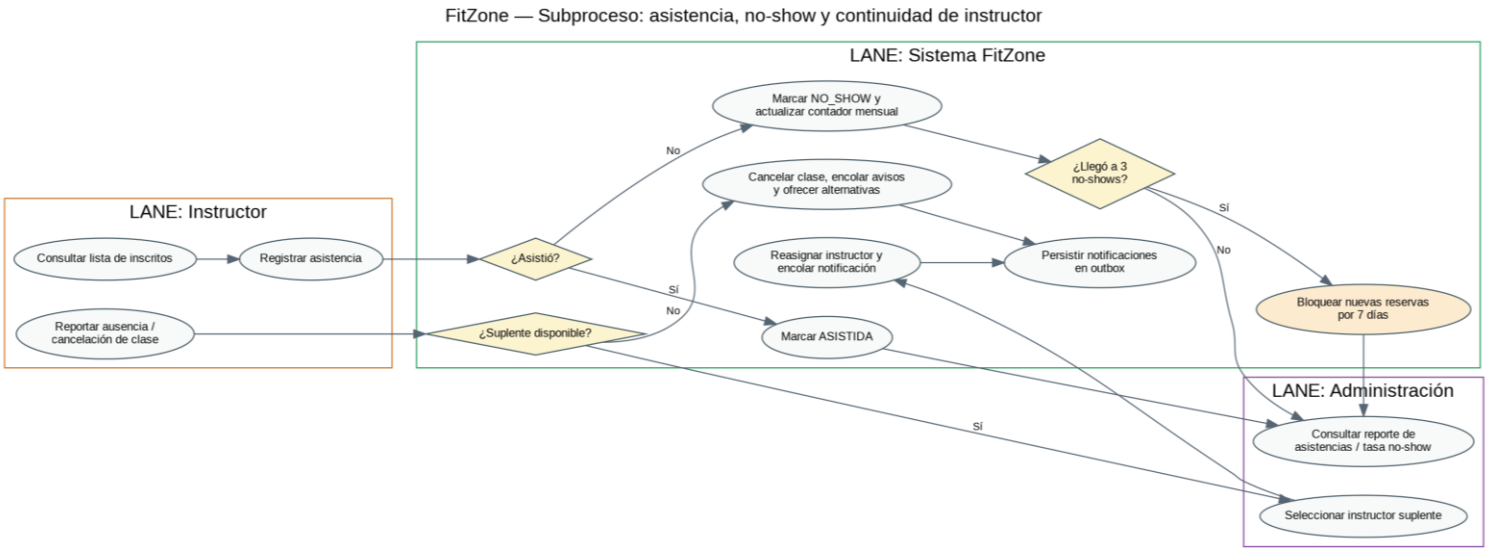
Reserva, cancelación y lista de espera FIFO



Contratación, vigencia y acceso físico



Asistencia, No Show y Continuidad de Instructor



Los diagramas BPMN complementarios permiten profundizar en los procesos específicos del sistema FitZone. Mientras que el diagrama general representa el funcionamiento global del negocio, los diagramas adicionales detallan actividades importantes como la gestión de reservas mediante una lista de espera FIFO, la validación de membresías para el control de acceso y el registro de asistencia por parte de los instructores. En conjunto, estos modelos proporcionan una visión más completa de los procedimientos que posteriormente se implementan en la lógica de negocio del sistema.

Glosario Oficial de Dominio (Lenguaje Ubicuo)

Término	Definición en el dominio de FitZone	Representación en código
Socio (Actor)	Cliente registrado que adquiere membresías, ingresa a las instalaciones y reserva clases.	Aggregate Root Socio
Plan Deportivo (Recurso)	Producto o servicio del catálogo con precio, capacidad o stock y estado de habilitación.	Entidad PlanDeportivo
Operación de Membresía	Transacción mediante la cual un socio contrata uno o más planes deportivos.	Aggregate Root OperacionMembresia
Vigencia	Periodo contractual que determina si una membresía está activa o vencida.	Value Object Vigencia
Dinero (Money)	Representación monetaria exacta mediante Decimal.	Value Object Money

Clase Programada	Sesión deportiva con disciplina, horario, instructor y capacidad máxima.	Aggregate Root ClaseProgramada
Reserva	Compromiso de un socio para ocupar un lugar en una clase.	Entidad interna Reserva
Lista de Espera	Cola ordenada por llegada para asignar lugares disponibles.	Colección en ClaseProgramada
Cut-off Time	Anticipación mínima permitida para cancelar sin penalización.	Regla en ClaseProgramada
No-Show	Inasistencia de un socio con reserva confirmada.	Método registrar_no_show()
Bloqueo Disciplinario	Restricción temporal para reservar clases por acumular inasistencias.	Estado en Socio
Drop-in	Inscripción extraordinaria de pago único a una clase fuera del plan.	ServicioDeInscripcionExtraordinaria
Membresía	Contrato que permite utilizar determinados servicios durante una vigencia definida.	Concepto gestionado por OperacionMembresia
Instructor	Persona asignada para impartir una clase y registrar la asistencia.	Atributo de ClaseProgramada
Capacidad máxima	Límite de reservas confirmadas permitido en una clase.	Atributo de ClaseProgramada
Cupo disponible	Cantidad de lugares libres en una clase.	Propiedad calculada de ClaseProgramada
Acceso físico	Decisión de permitir o denegar el ingreso al gimnasio.	ControlAccesoService
Check-in	Registro de presentación del socio a una sesión o instalación.	Registro de asistencia
Specification	Objeto que encapsula una regla de negocio verificable y combinable.	Patrón Specification
Máquina de estados	Modelo que establece los estados y transiciones válidas de una entidad.	Máquina de estados del dominio

2. Fase de arquitectura

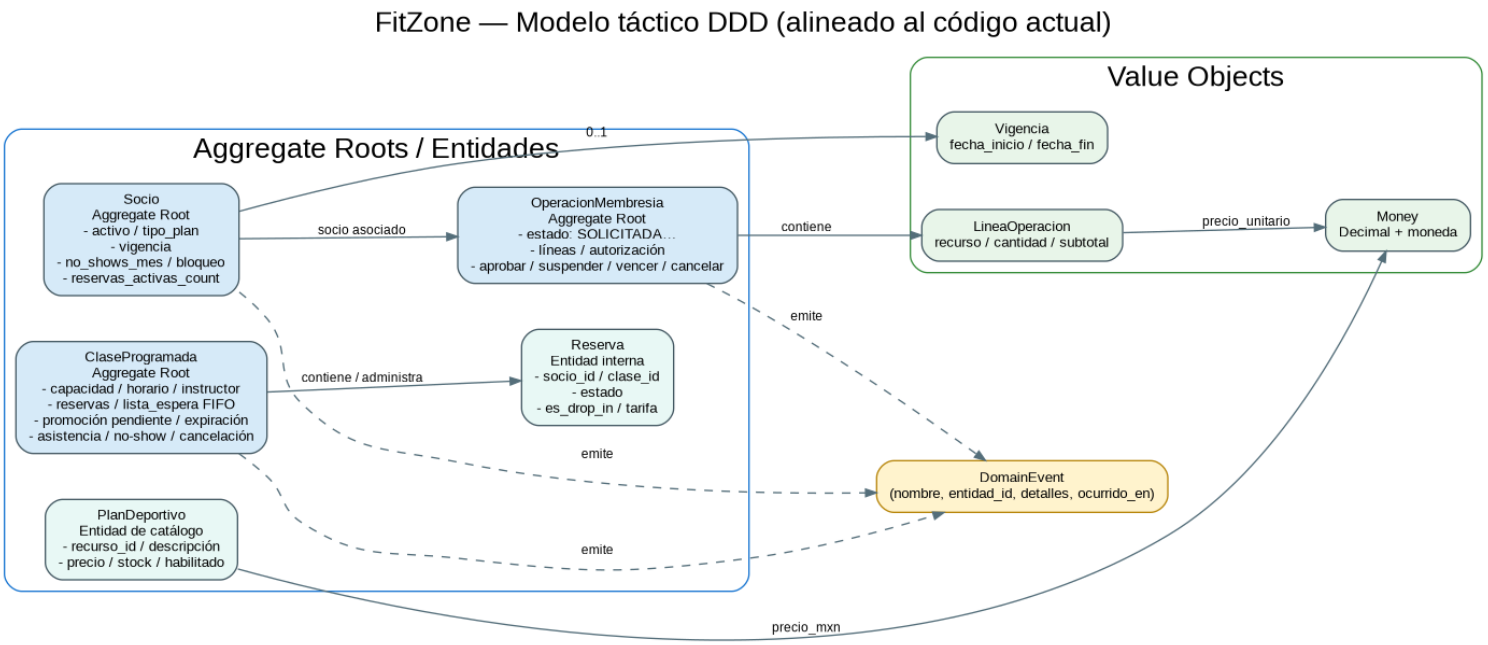
2.1 Diseño guiado por el dominio (DDD)

La arquitectura se diseñó alrededor del lenguaje y las reglas de FitZone. El dominio concentra los conceptos y comportamientos del negocio; la capa de aplicación coordina los casos de uso; infraestructura implementa persistencia y adaptadores; y la API expone las operaciones a los clientes.

En el modelo táctico actualizado, las raíces de agregado identificadas en el código son Socio, OperacionMembresia y ClasePrograma. Reserva es una entidad interna administrada por ClasePrograma, mientras que PlanDeportivo funciona como entidad de catálogo. Esta distinción evita presentar cada objeto como agregado independiente sin que el código lo respalde.

Elemento DDD	Responsabilidad en FitZone
Socio (agregado)	Representa al cliente del gimnasio y sus datos/condiciones relevantes.
OperacionMembresia (agregado)	Controla operaciones relacionadas con la contratación o modificación de membresías.
ClasePrograma (agregado)	Gestiona la clase y sus reservas internas, capacidad y estados.
Reserva (entidad interna)	Representa la participación solicitada/confirmada de un socio en una clase.
PlanDeportivo (catálogo)	Describe las modalidades o planes disponibles.
Money, Vigencia, LineaOperacion (Value Objects)	Representan valores del dominio con significado y validaciones propias.

DDD

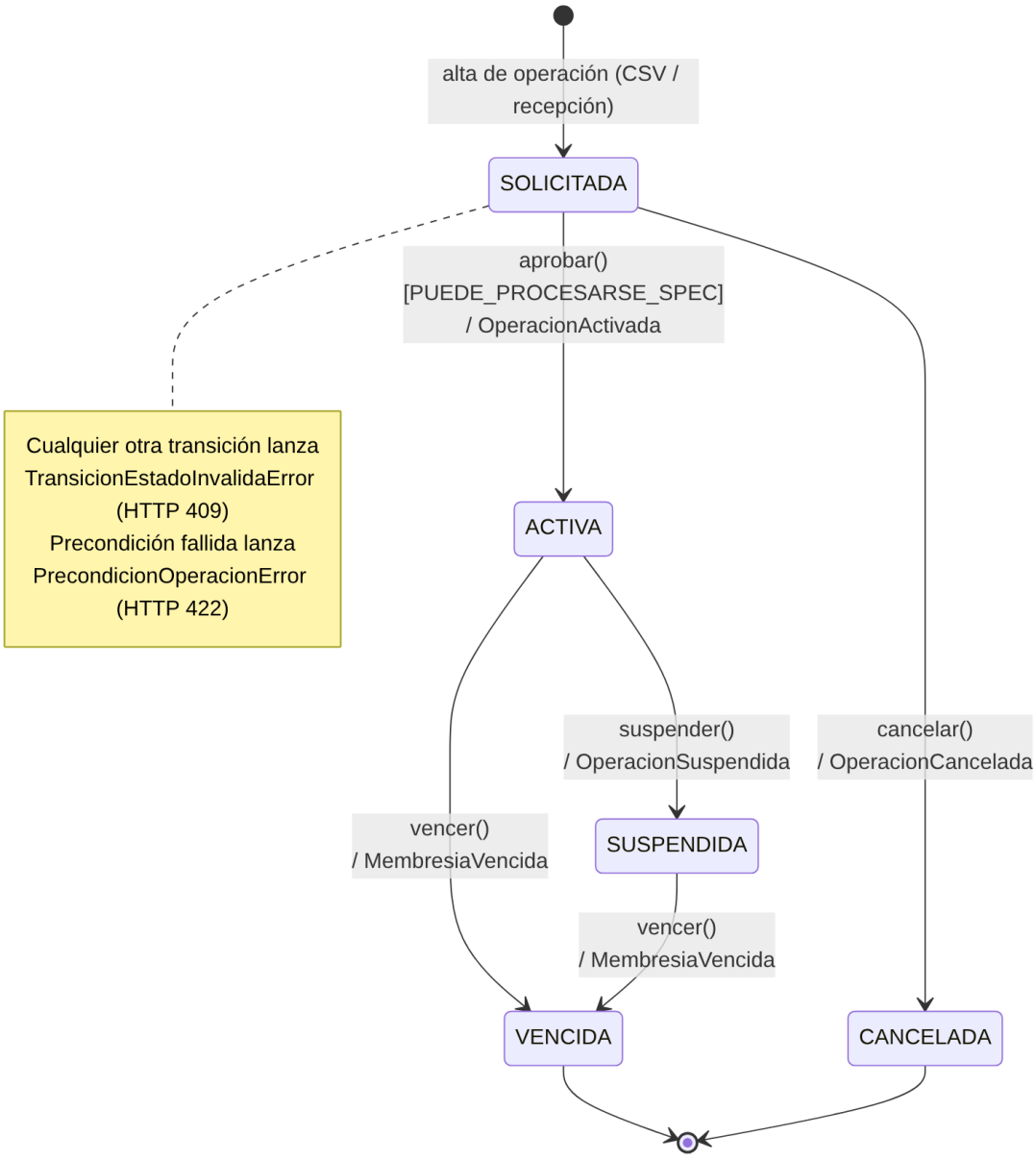


2.2 Specification y State Machine

El patrón Specification encapsula condiciones del negocio en objetos reutilizables que pueden combinarse mediante operaciones booleanas AND, OR y NOT. Esto permite expresar reglas de elegibilidad y validación sin dispersar condicionales por todos los servicios.

La máquina de estados controla transiciones válidas de los objetos que tienen ciclo de vida. En lugar de permitir cambios arbitrarios, comprueba el estado actual y la transición solicitada, y puede registrar eventos de dominio asociados al resultado. El repositorio incluye el diagrama de estados y las pruebas correspondientes.

Máquina de Estados



2.3 Servicios, puertos y principios SOLID

La capa de aplicación contiene servicios para membresías, reservas, asistencia, control de acceso y consultas de reportes. Los puertos describen dependencias necesarias —repositorios, Unit of Work, notificador, torniquete y pasarela de pago— mientras que los adaptadores concretos se ubican en infraestructura.

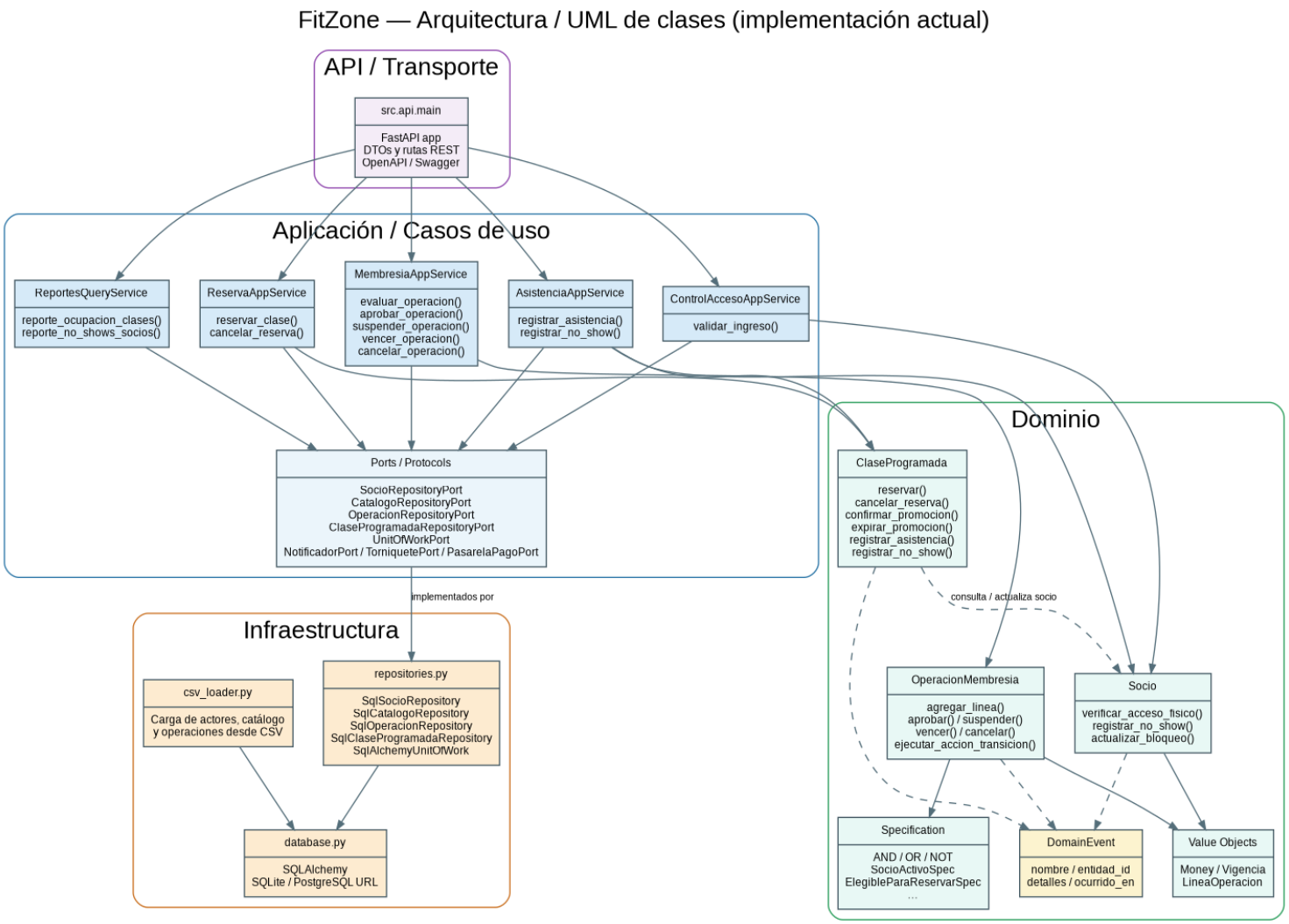
Principio	Aplicación en el proyecto
S — Responsabilidad única	Separación de dominio, casos de uso, persistencia, API y tareas programadas.
O — Abierto/cerrado	Reglas encapsuladas y extensibles mediante especificaciones y contratos.
L — Sustitución de Liskov	Los adaptadores deben respetar los contratos definidos por los puertos.
I — Segregación de interfaces	Puertos enfocados en las capacidades que requiere cada servicio.
D — Inversión de dependencias	La aplicación depende de abstracciones y no de detalles concretos de infraestructura.

La aplicación de SOLID se evidencia en la organización del código y los contratos; no implica que todas las integraciones externas estén conectadas a proveedores reales.

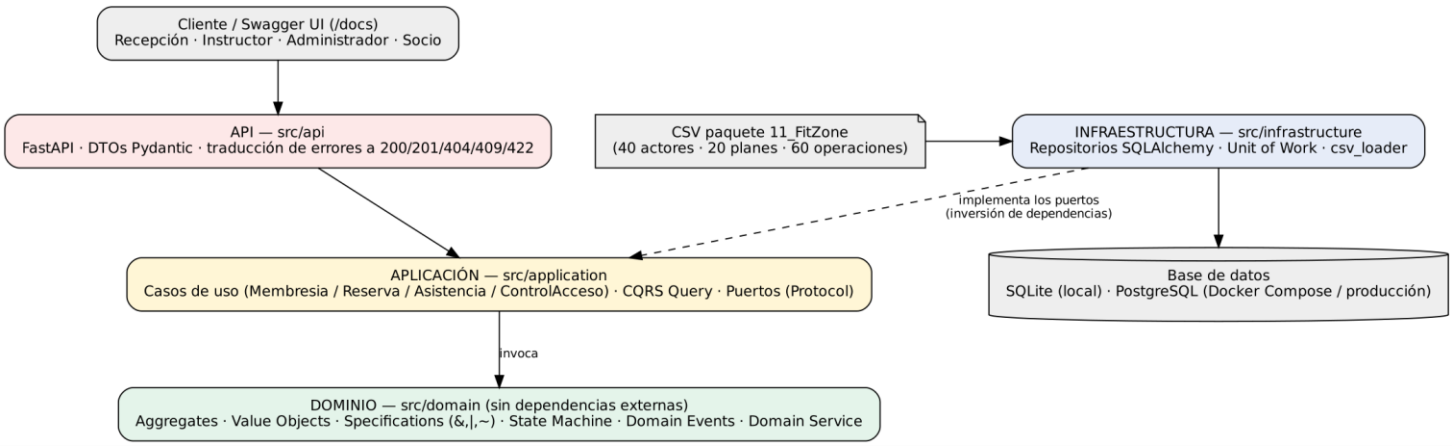
2.4 UML, arquitectura por capas y ADRs

Los diagramas UML representan clases y relaciones del dominio y de la solución. El diagrama de arquitectura por capas muestra cómo se separan las responsabilidades y por dónde circulan las dependencias. Los ADRs documentan decisiones técnicas relevantes, su contexto, la alternativa seleccionada y sus consecuencias.

UML


FitZone — Arquitectura / UML de clases (implementación actual)
API / Transporte
src.api.main
FastAPI app
DTOs y rutas REST
OpenAPI / Swagger
Aplicación / Casos de uso
ReportesQueryService
reporte_ocupacion_clases()
reporte_no_shows_socios()
ReservaAppService
reservar_clase()
cancelar_reserva()
MembresiaAppService
evaluar_operacion()
aprobar_operacion()
suspender_operacion()
vencer_operacion()
cancelar_operacion()
AsistenciaAppService
registrar_asistencia()
registrar_no_show()
ControlAccesoAppService
validar_ingreso()
Ports / Protocols
SocioRepositoryPort
CatalogoRepositoryPort
OperacionRepositoryPort
ClaseProgramadaRepositoryPort
UnitOfWorkPort
NotificadorPort / TorniquetePort / PasarelaPagoPort
implementados por
Infraestructura
csv_loader.py
Carga de actores, catálogo y operaciones desde CSV
repositories.py
SqlSocioRepository
SqlCatalogoRepository
SqlOperacionRepository
SqlClaseProgramadaRepository
SqlAlchemyUnitOfWork
database.py
SQLAlchemy
SQLite / PostgreSQL URL
Dominio
ClaseProgramada
reservar()
cancelar_reserva()
confirmar_promocion()
expirar_promocion()
registrar_asistencia()
registrar_no_show()
consulta / actualiza socio
OperacionMembresia
agregar_linea()
aprobar() / suspender()
vencer() / cancelar()
ejecutar_accion_transicion()
Socio
verificar_acceso_fisico()
registrar_no_show()
actualizar_bloqueo()
Specification
AND / OR / NOT
SocioActivoSpec
ElegibleParaReservarSpec
...
DomainEvent
nombre / entidad_id
detalles / ocurrido_en
Value Objects
Money / Vigencia
LineaOperacion

Arquitectura de Capas



Architecture Decision Records (ADRs)

ADR-01: Adoptar Domain Model Puro con Arquitectura Hexagonal

Estado: Aceptada

Contexto: FitZone posee invariantes fuertemente interdependientes (máquina de estados de membresías, *Specifications* booleanas, aforo máximo con lista de espera FIFO y bloqueo disciplinario por 3 *no-shows*).

Alternativas consideradas: *Transaction Script* (descartado por duplicación de validaciones y riesgo de condiciones de carrera) y *Table Module* (descartado por acoplamiento a tablas relacionales).

Decisión: Implementar un *Domain Model* puro en src/domain/ desacoplado de la persistencia (src/infrastructure/) y del transporte HTTP (src/api/).

Consecuencias: El núcleo de negocio se puede probar de forma aislada y reutilizar desde scripts por lotes (CSVs) o la API REST FastAPI. La cobertura global y los escenarios cubiertos se reportan en los resultados de pytest; no se afirma cobertura absoluta.

ADR-02: Patrón Unit of Work para delimitar transacciones compuestas

Estado: Aceptada

Contexto: En CU-03 (Cancelación anticipada), liberar el cupo de un socio y promover automáticamente al primer socio de la lista de espera FIFO deben confirmarse como una única operación atómica. Lo mismo aplica al aprobar una OperacionMembresia descontando stock del catálogo.

Alternativas consideradas: Ejecutar commit() individuales por cada tabla modificada (riesgo de dejar aforos corruptos o lugares huérfanos ante un fallo intermedio).

Decisión: Implementar control transaccional explícito (commit / rollback) en la capa de repositorio SQLAlchemy, asegurando que cualquier excepción de dominio revierta íntegramente la sesión.

Consecuencias: El Unit of Work centraliza commit/rollback y cuenta con pruebas de persistencia y reversión; la concurrencia requiere controles adicionales.

ADR-03: Aplicar CQRS Básico en Consultas de Disponibilidad y Analítica

Estado: Aceptada

Contexto: Mientras los *Commands* de reserva, cancelación, aprobación y sanción exigen cargar Aggregates completos y verificar invariantes, las consultas masivas de disponibilidad de clases (US-002) y reportes administrativos de ocupación y *no-shows* (US-014, US-015) son de solo lectura.

Decisión: Separar los métodos de consulta rápida (ReportesQueryService) de los servicios transaccionales de comando (MembresiaAppService, ReservaAppService).

Consecuencias: Lecturas rápidas y desacopladas sin comprometer la protección de invariantes en la escritura.

ADR-04: Uso de Decimal y Specification Pattern para Reglas Comerciales y Paquete de Datos Oficial

Estado: Aceptada

Contexto: El paquete oficial de pruebas de FitZone (11_FitZone.zip) requiere evaluar 60 operaciones (FIT-O001 a FIT-O060), 120 casos de prueba (FIT-T001..060 y FIT-B001..060) y 20 transiciones de máquina de estados (FIT-S001..020) con importes en pesos mexicanos (MXN).

Decisión: Utilizar el Value Object Money basado exclusivamente en decimal.Decimal y modelar validar_operacion y evaluar_beneficio mediante composición de objetos Specification (&, |, ~).

Consecuencias: Cero errores de redondeo binario y trazabilidad directa 1 a 1 entre las reglas del CSV oficial, el BRD y los tests automatizados.

3. Fase de implementación

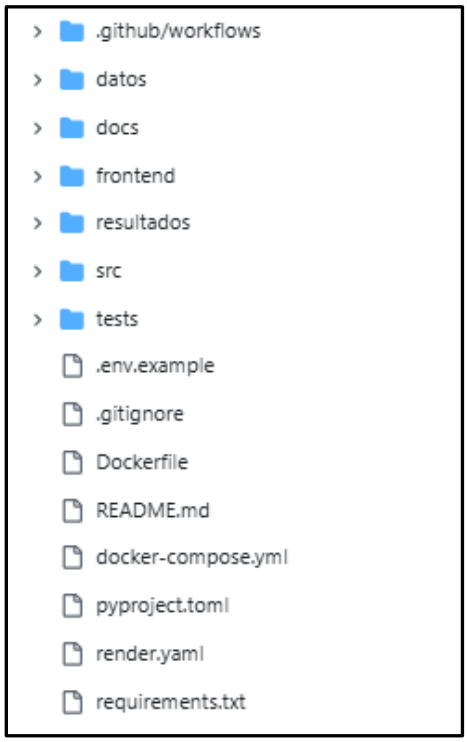
3.1 Estructura del repositorio

El repositorio se organizó siguiendo la estructura mínima solicitada por la guía y se amplió con carpetas para documentación, datos, resultados, interfaz y tareas programadas. La estructura relevante es:

```
fitzone-pia-equipo-11/
├── .github/workflows/ci.yml
├── datos/
│   ├── actores.csv
│   ├── catalogo.csv
│   ├── operaciones.csv
│   ├── detalle_operaciones.csv
│   ├── casos_prueba.csv
│   └── transiciones_prueba.csv
├── docs/
│   ├── BRD.md
│   ├── architecture/
│   │   ├── ARCHITECTURE_AND_ADRS.md
│   │   └── diagrams/
│   ├── phase2/
│   │   ├── MATRIZ_TRAZABILIDAD_FITZONE.md
│   │   └── README_FASE_2.md
│   └── api/openapi.json
├── frontend/
│   ├── index.html
│   ├── styles.css
│   └── app.js
├── src/
│   ├── api/
│   ├── application/
│   ├── domain/
│   ├── infrastructure/
│   └── jobs/
├── tests/
├── resultados/
├── Dockerfile
├── docker-compose.yml
├── .env.example
├── requirements.txt
├── pyproject.toml
├── render.yaml
└── README.md
```

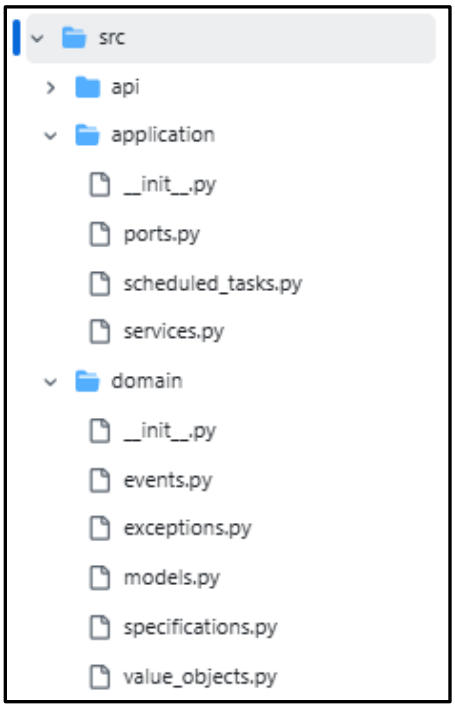
Carpeta/archivo	Función
docs/	BRD, arquitectura, diagramas, decisiones, matriz de trazabilidad y especificación OpenAPI.
src/domain/	Entidades, agregados, Value Objects, especificaciones, eventos y excepciones del dominio.
src/application/	Servicios, casos de uso, puertos y coordinación de operaciones.
src/infrastructure/	Persistencia, repositorios, base de datos y carga de datos.
src/api/	Aplicación FastAPI y endpoints REST.
src/jobs/	Worker para tareas periódicas del sistema.
frontend/	Interfaz web que consume la API.

tests/	Pruebas unitarias, de integración, escenarios CSV y tareas programadas.
datos/ y resultados/	Datos de entrada para escenarios y archivos de resultados procesados.
.github/workflows/	Pipeline automatizado de validación y pruebas.



3.2 Dominio y servicios de aplicación

El código de dominio concentra los modelos y reglas principales. Los servicios de aplicación coordinan acciones como operaciones de membresía, reservas, asistencia, control de acceso y reportes. Esta separación permite que la lógica de negocio pueda probarse sin depender directamente de la interfaz web o de la base de datos.

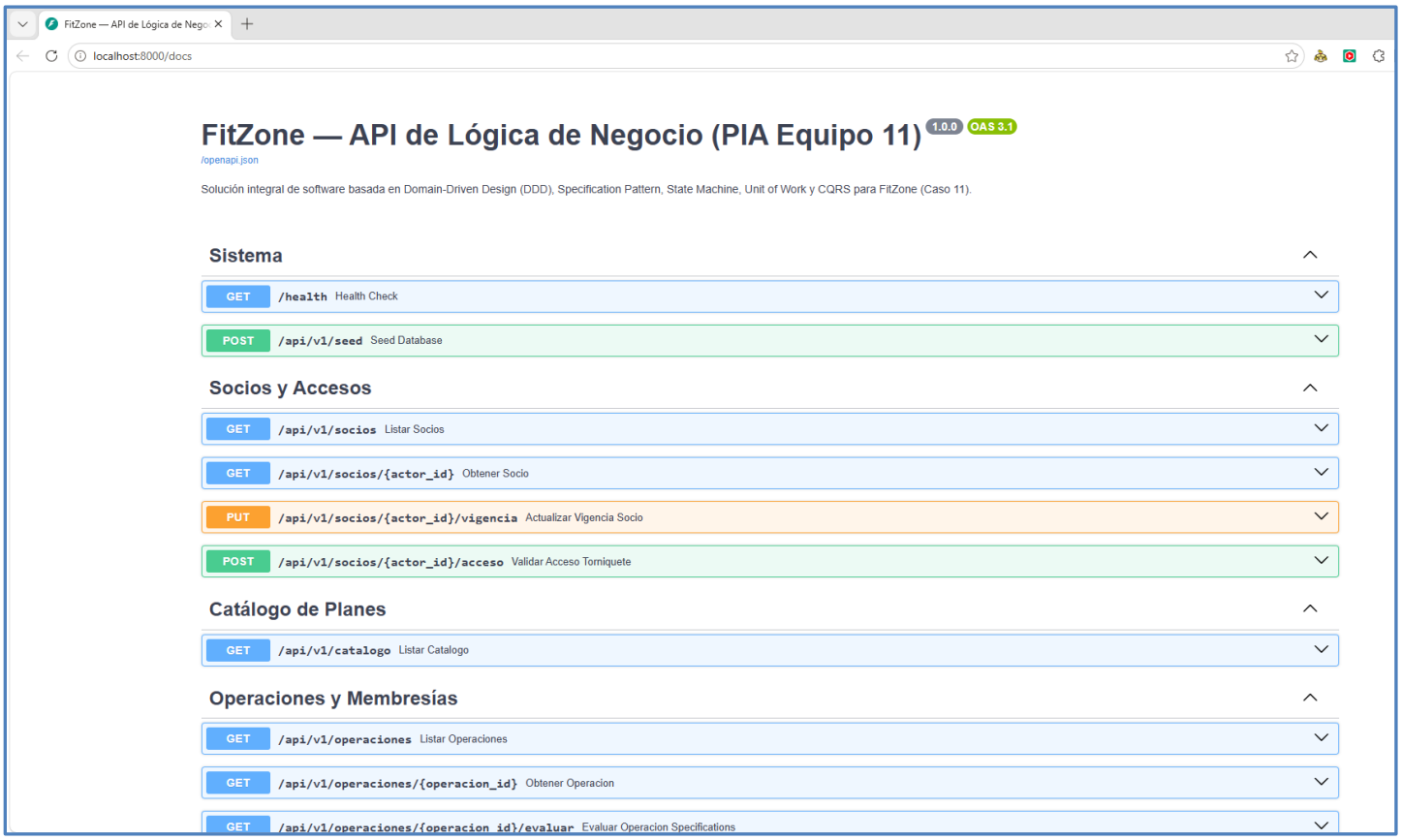


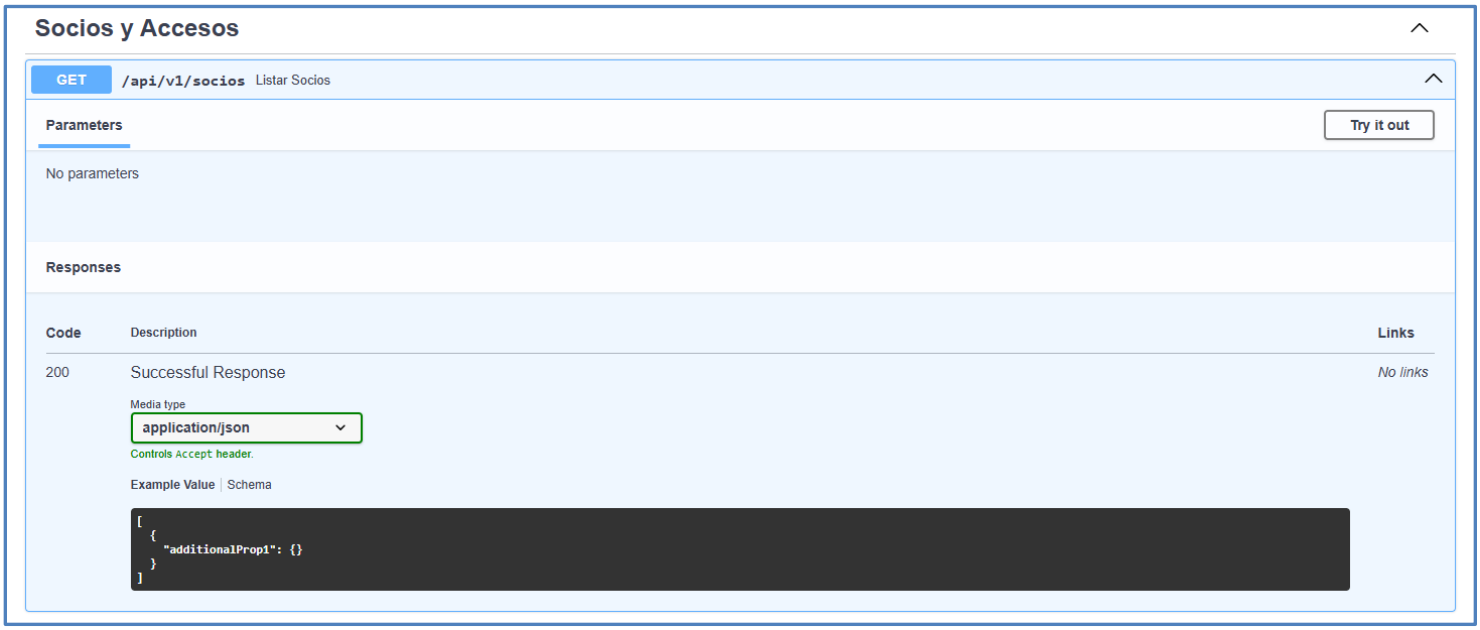
3.3 Persistencia y API REST

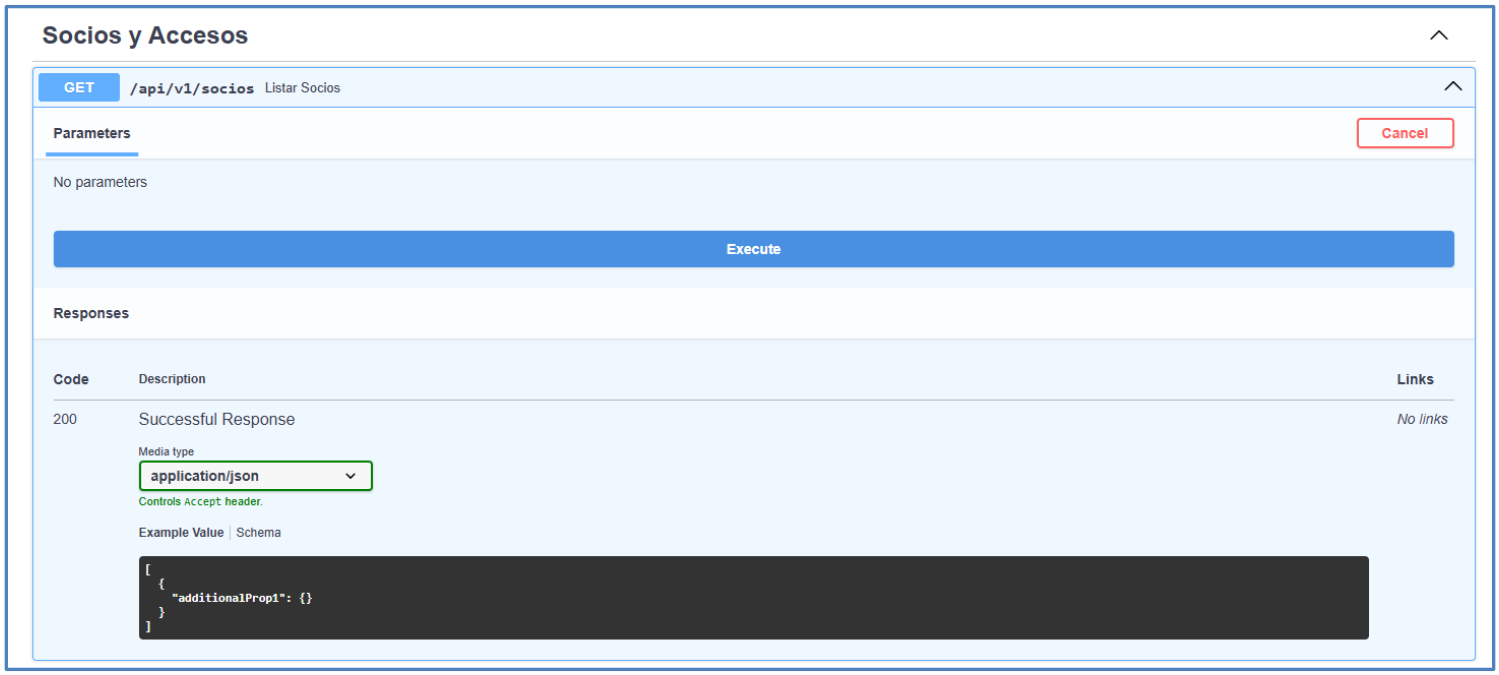
La solución utiliza FastAPI para exponer operaciones mediante endpoints REST y SQLAlchemy para la persistencia relacional. La documentación OpenAPI permite inspeccionar los recursos, parámetros y respuestas desde Swagger UI. La base de datos PostgreSQL se utiliza en el entorno Docker Compose local.

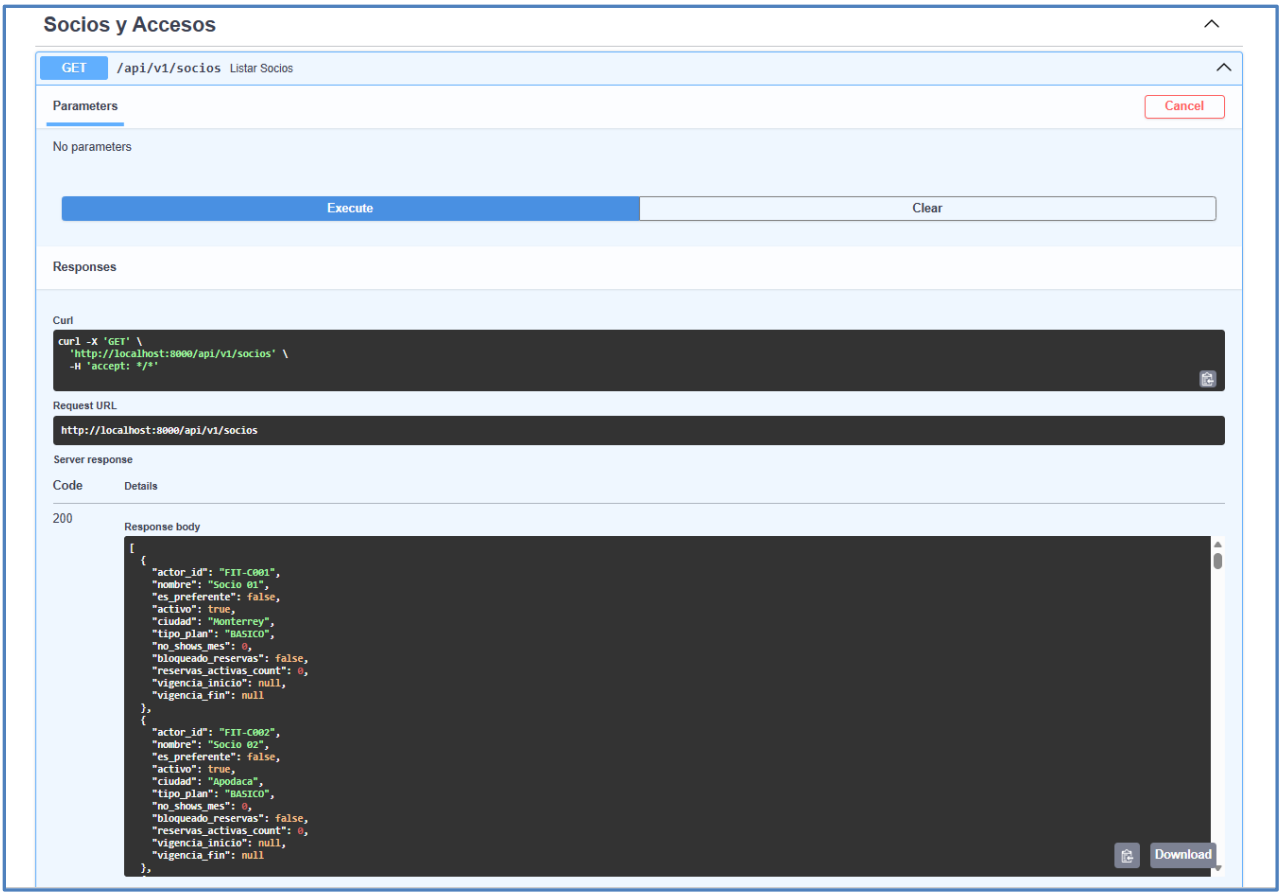
La API incorpora operaciones para consultar y administrar socios, membresías, clases y reservas, además de operaciones de asistencia, acceso y reportes según los casos implementados. La interfaz web consume la API para presentar el panel y las pantallas funcionales; no se describe como una maqueta desconectada.

Back-end

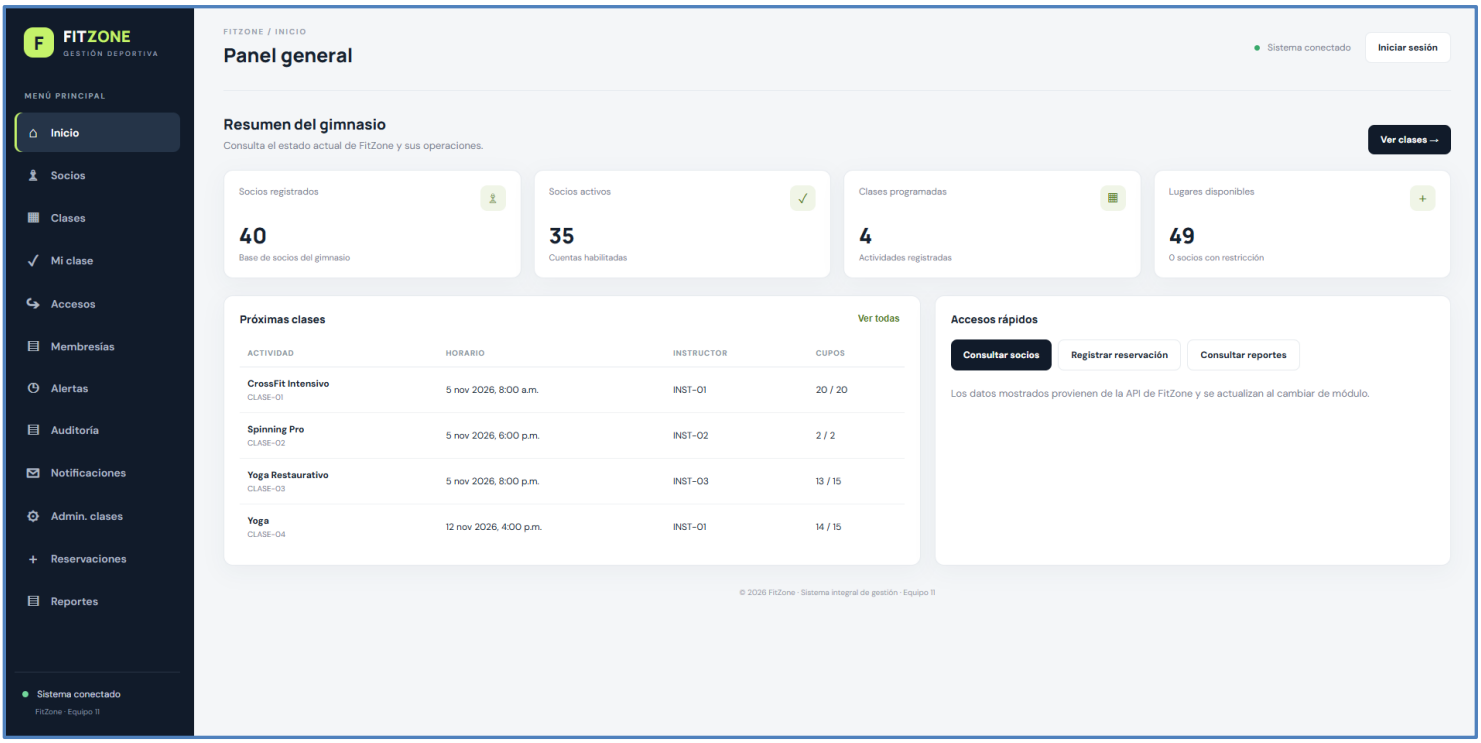


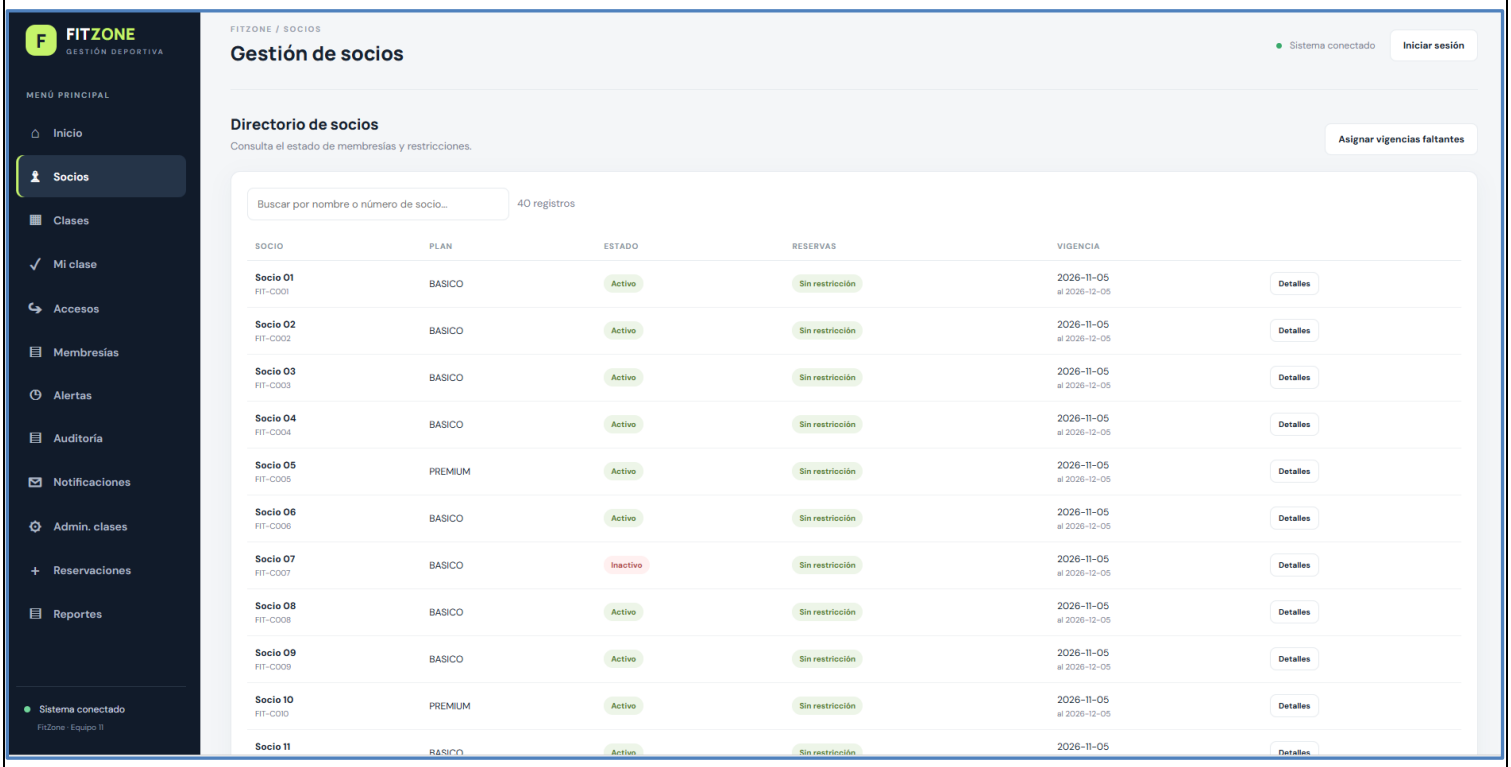

Socios y Accesos
GET
/api/v1/socios
Listar Socios
Parameters
Try it out
No parameters
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
[
{
"additionalProp1": {}
}
]

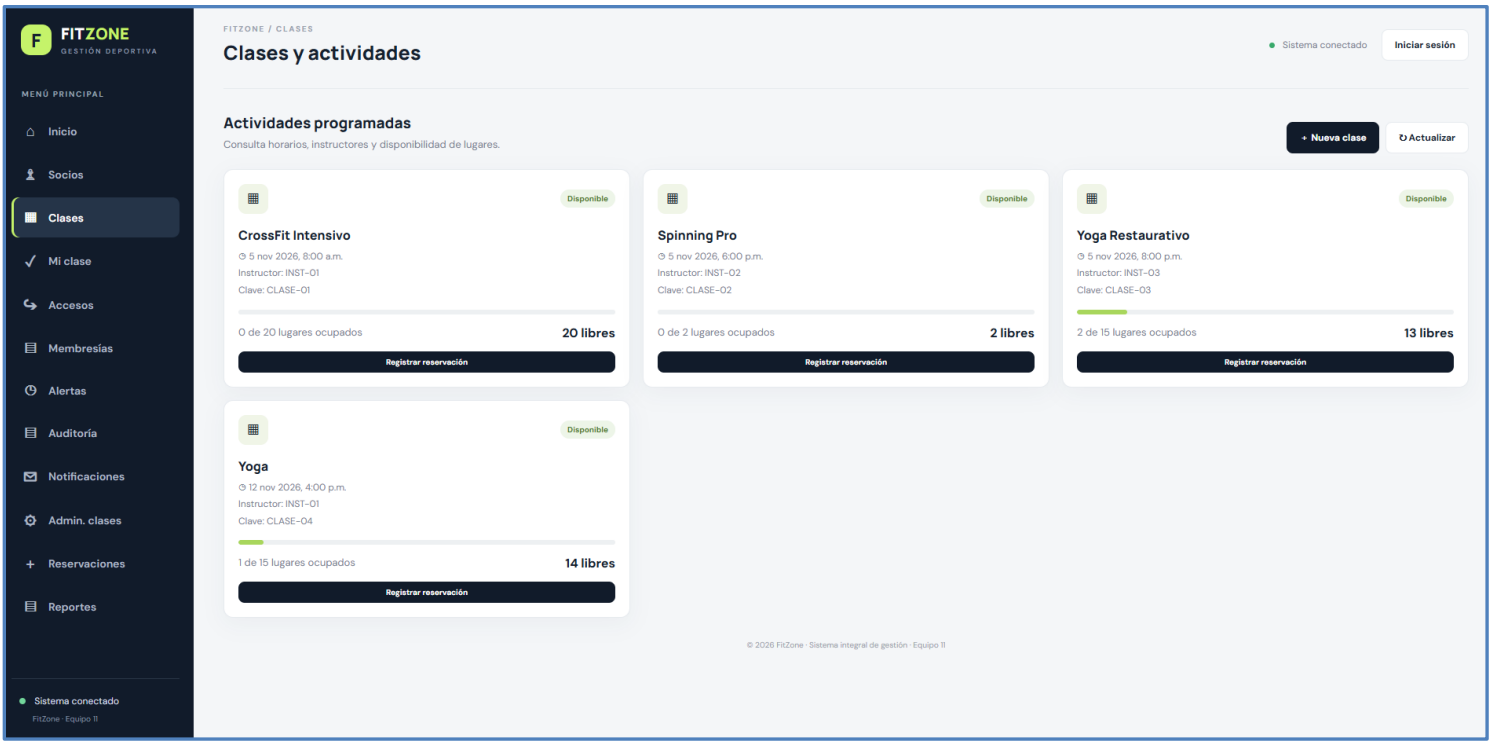

Socios y Accesos
GET
/api/v1/socios
Listar Socios
Parameters
Cancel
No parameters
Execute
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
[
{
"additionalProp1": {}
}
]

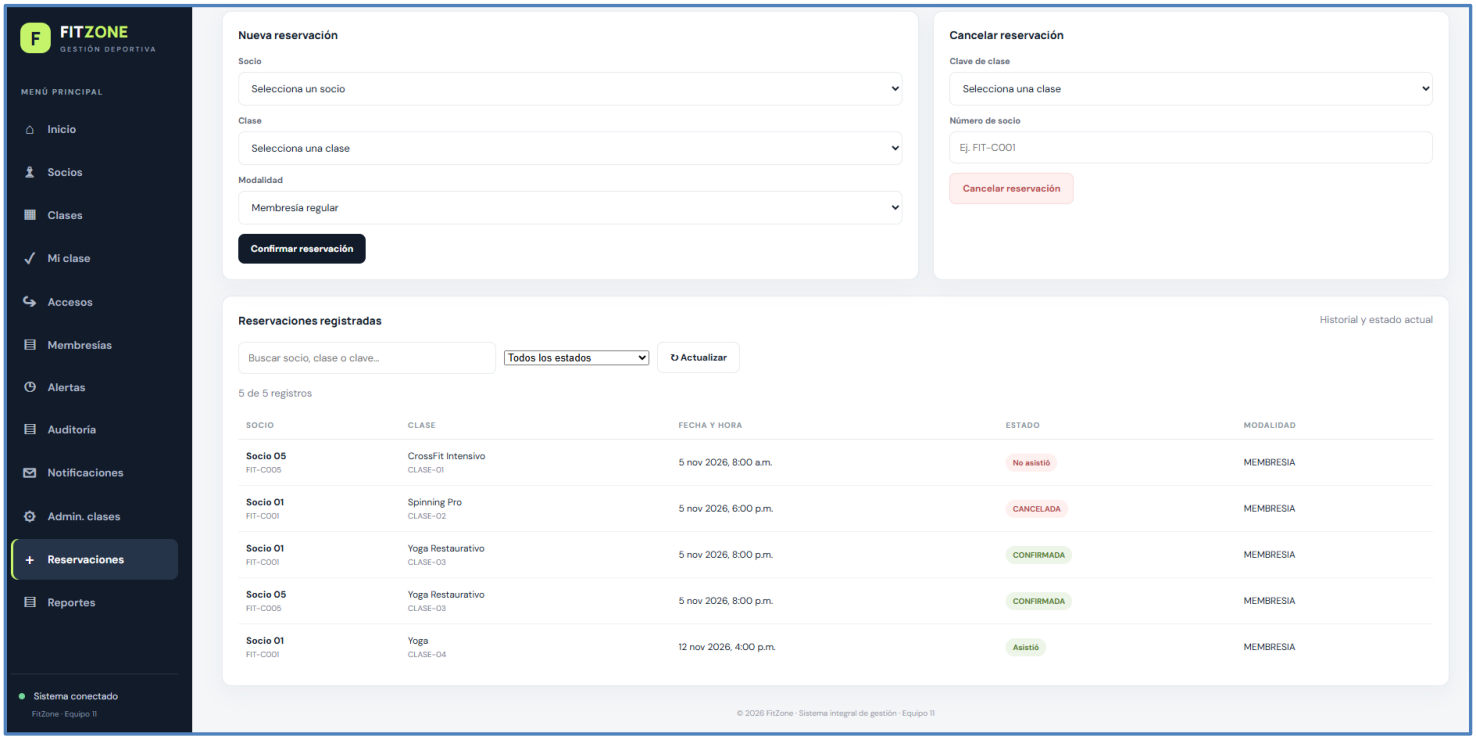

Socios y Accesos
GET /api/v1/socios Listar Socios
Parameters
Cancel
No parameters
Execute
Clear
Responses
Curl
curl -X 'GET' \
'http://localhost:8000/api/v1/socios' \
-H 'accept: */*'
Request URL
http://localhost:8000/api/v1/socios
Server response
Code
Details
200
Response body
[
{
"actor_id": "FIT-C001",
"nombre": "Socio 01",
"es_preferente": false,
"activo": true,
"ciudad": "Monterrey",
"tipo_plan": "BASICO",
"no_shows_mes": 0,
"bloqueado_reservas": false,
"reservas_activas_count": 0,
"vigencia_inicio": null,
"vigencia_fin": null
},
{
"actor_id": "FIT-C002",
"nombre": "Socio 02",
"es_preferente": false,
"activo": true,
"ciudad": "Apodaca",
"tipo_plan": "BASICO",
"no_shows_mes": 0,
"bloqueado_reservas": false,
"reservas_activas_count": 0,
"vigencia_inicio": null,
"vigencia_fin": null
},
Download

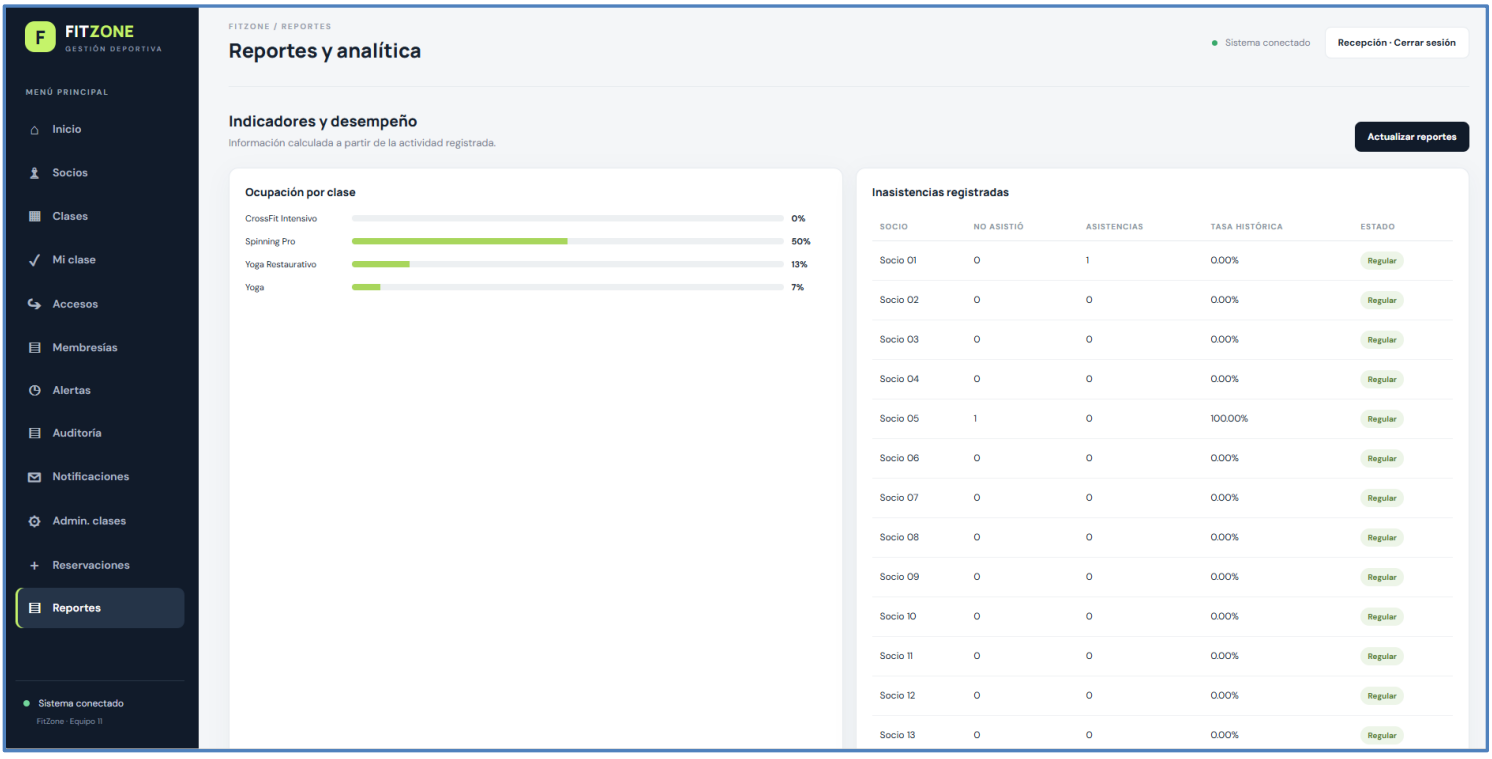
Front-end


FITZONE
GESTIÓN DEPORTIVA
MENÚ PRINCIPAL
Inicio
Socios
Clases
Mi clase
Accesos
Membresías
Alertas
Auditoría
Notificaciones
Admin. clases
Reservaciones
Reportes
Sistema conectado
FitZone · Equipo 11
FITZONE / INICIO
Panel general
Sistema conectado
Iniciar sesión
Resumen del gimnasio
Consulta el estado actual de FitZone y sus operaciones.
Ver clases →
Socios registrados
40
Base de socios del gimnasio
Socios activos
35
Cuentas habilitadas
Clases programadas
4
Actividades registradas
Lugares disponibles
49
0 socios con restricción
Próximas clases
Ver todas
ACTIVIDAD
HORARIO
INSTRUCTOR
CUPOS
CrossFit Intensivo
CLASE-01
5 nov 2026, 8:00 a.m.
INST-01
20 / 20
Spinning Pro
CLASE-02
5 nov 2026, 6:00 p.m.
INST-02
2 / 2
Yoga Restaurativo
CLASE-03
5 nov 2026, 8:00 p.m.
INST-03
13 / 15
Yoga
CLASE-04
12 nov 2026, 4:00 p.m.
INST-01
14 / 15
Accesos rápidos
Consultar socios
Registrar reservación
Consultar reportes
Los datos mostrados provienen de la API de FitZone y se actualizan al cambiar de módulo.
© 2026 FitZone · Sistema integral de gestión · Equipo 11

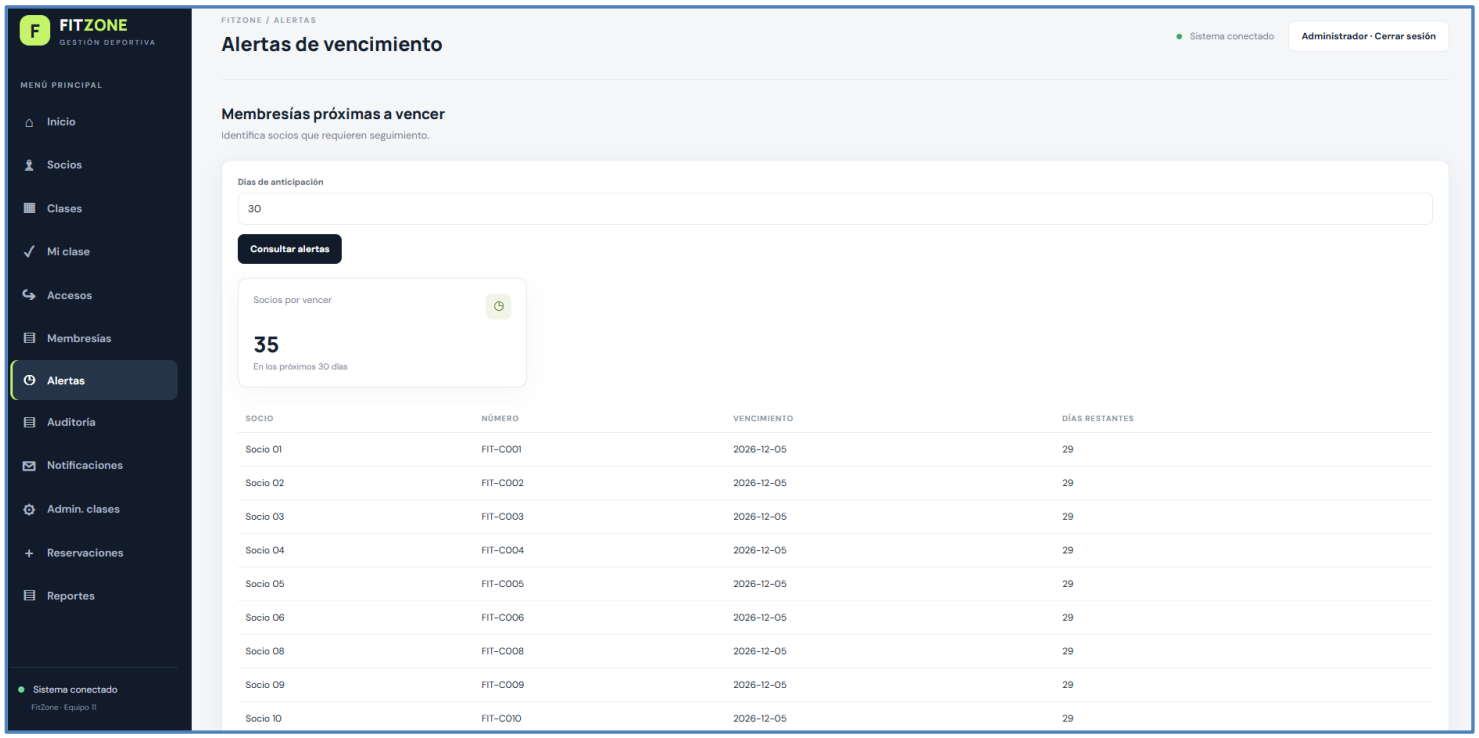

FITZONE
GESTIÓN DEPORTIVA
MENÚ PRINCIPAL
Inicio
Socios
Clases
Mi clase
Accesos
Membresías
Alertas
Auditoría
Notificaciones
Admin. clases
Reservaciones
Reportes
Sistema conectado
FitZone · Equipo 11
FITZONE / SOCIOS
Gestión de socios
Sistema conectado
Iniciar sesión
Directorio de socios
Consulta el estado de membresías y restricciones.
Asignar vigencias faltantes
Buscar por nombre o número de socio...
40 registros
SOCIO
PLAN
ESTADO
RESERVAS
VIGENCIA
Socio 01 FIT-C001 BASICO Activo Sin restricción 2026-11-05 al 2026-12-05 Detalles
Socio 02 FIT-C002 BASICO Activo Sin restricción 2026-11-05 al 2026-12-05 Detalles
Socio 03 FIT-C003 BASICO Activo Sin restricción 2026-11-05 al 2026-12-05 Detalles
Socio 04 FIT-C004 BASICO Activo Sin restricción 2026-11-05 al 2026-12-05 Detalles
Socio 05 FIT-C005 PREMIUM Activo Sin restricción 2026-11-05 al 2026-12-05 Detalles
Socio 06 FIT-C006 BASICO Activo Sin restricción 2026-11-05 al 2026-12-05 Detalles
Socio 07 FIT-C007 BASICO Inactivo Sin restricción 2026-11-05 al 2026-12-05 Detalles
Socio 08 FIT-C008 BASICO Activo Sin restricción 2026-11-05 al 2026-12-05 Detalles
Socio 09 FIT-C009 BASICO Activo Sin restricción 2026-11-05 al 2026-12-05 Detalles
Socio 10 FIT-C010 PREMIUM Activo Sin restricción 2026-11-05 al 2026-12-05 Detalles
Socio 11 BASICO Activo Sin restricción 2026-11-05 Detalles

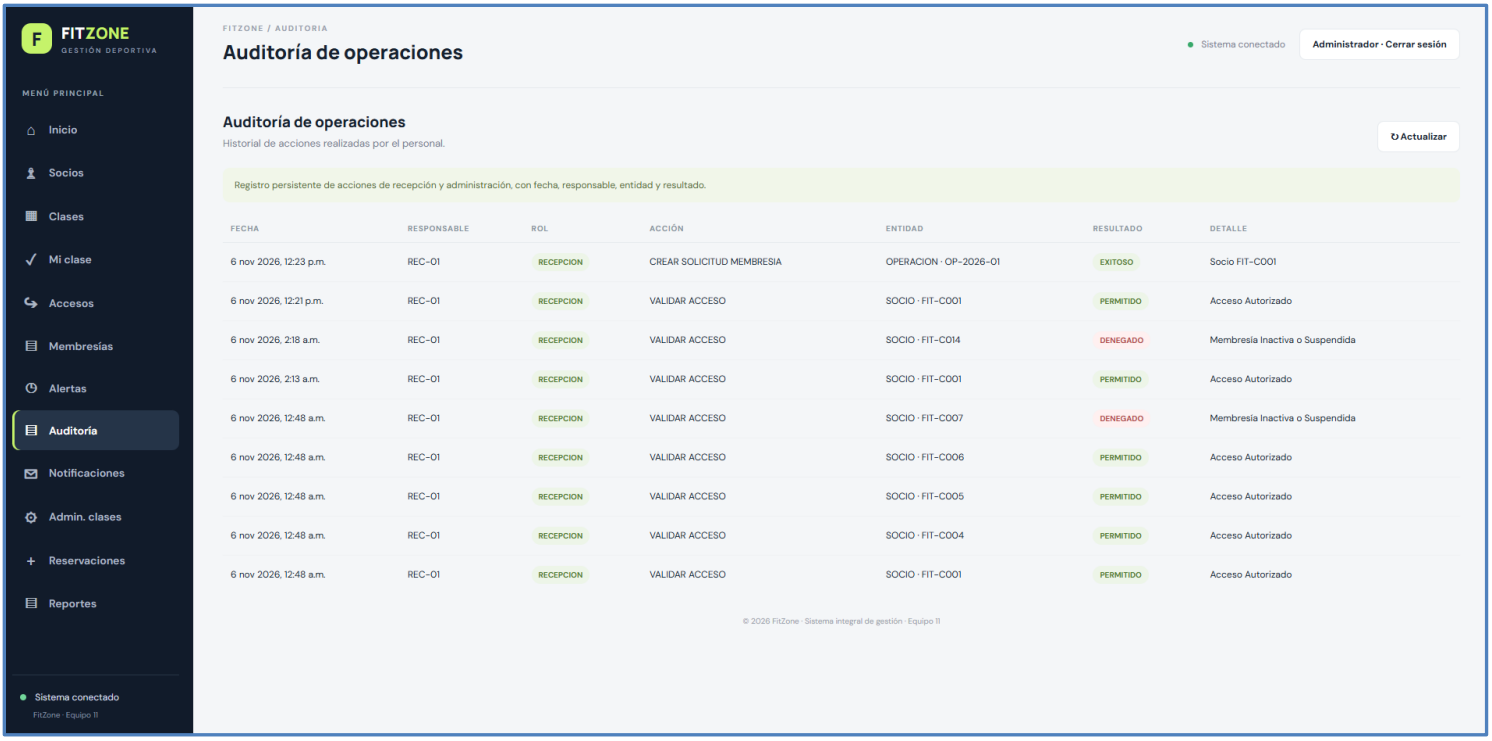

FITZONE
GESTIÓN DEPORTIVA
MENÚ PRINCIPAL
Inicio
Socios
Clases
Mi clase
Accesos
Membresías
Alertas
Auditoría
Notificaciones
Admin. clases
Reservaciones
Reportes
Sistema conectado
FitZone · Equipo 11
FITZONE / CLASES
Clases y actividades
Sistema conectado
Iniciar sesión
Actividades programadas
Consulta horarios, instructores y disponibilidad de lugares.
+ Nueva clase
Actualizar
Disponible
CrossFit Intensivo
5 nov 2026, 8:00 a.m.
Instructor: INST-01
Clave: CLASE-01
0 de 20 lugares ocupados
20 libres
Registrar reservación
Disponible
Spinning Pro
5 nov 2026, 6:00 p.m.
Instructor: INST-02
Clave: CLASE-02
0 de 2 lugares ocupados
2 libres
Registrar reservación
Disponible
Yoga Restaurativo
5 nov 2026, 8:00 p.m.
Instructor: INST-03
Clave: CLASE-03
2 de 15 lugares ocupados
13 libres
Registrar reservación
Disponible
Yoga
12 nov 2026, 4:00 p.m.
Instructor: INST-01
Clave: CLASE-04
1 de 15 lugares ocupados
14 libres
Registrar reservación
© 2026 FitZone · Sistema integral de gestión · Equipo 11

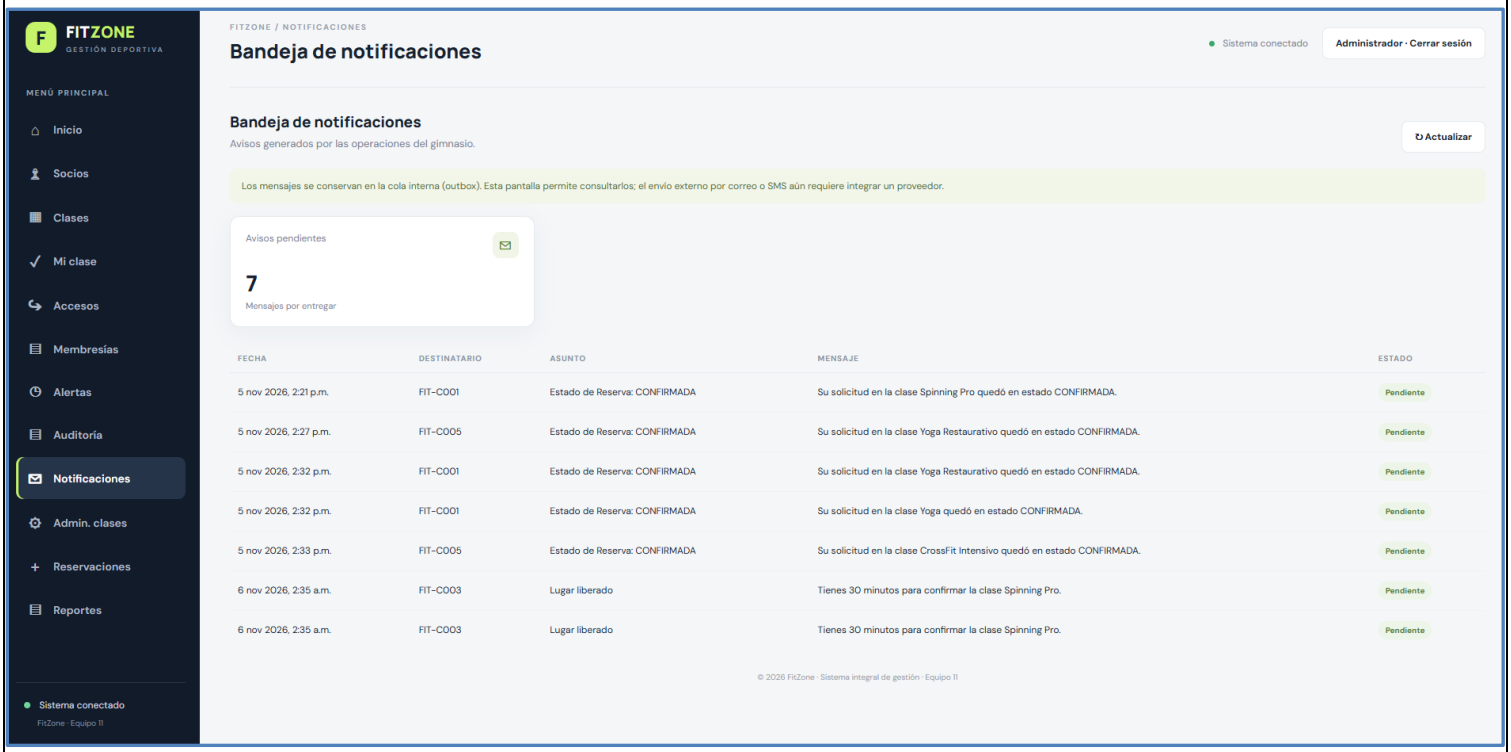

FITZONE
GESTIÓN DEPORTIVA
MENÚ PRINCIPAL
Inicio
Socios
Clases
Mi clase
Accesos
Membresías
Alertas
Auditoría
Notificaciones
Admin. clases
Reservaciones
Reportes
Sistema conectado
FitZone · Equipo 11
Nueva reservación
Socio
Selecciona un socio
Clase
Selecciona una clase
Modalidad
Membresía regular
Confirmar reservación
Cancelar reservación
Clave de clase
Selecciona una clase
Número de socio
Ej. FIT-C001
Cancelar reservación
Reservaciones registradas
Historial y estado actual
Buscar socio, clase o clave...
Todos los estados
↻ Actualizar
5 de 5 registros
SOCIO
CLASE
FECHA Y HORA
ESTADO
MODALIDAD
Socio 05
FIT-C005
CrossFit Intensivo
CLASE-01
5 nov 2026, 8:00 a.m.
No asistió
MEMBRESIA
Socio 01
FIT-C001
Spinning Pro
CLASE-02
5 nov 2026, 6:00 p.m.
CANCELADA
MEMBRESIA
Socio 01
FIT-C001
Yoga Restaurativo
CLASE-03
5 nov 2026, 8:00 p.m.
CONFIRMADA
MEMBRESIA
Socio 05
FIT-C005
Yoga Restaurativo
CLASE-03
5 nov 2026, 8:00 p.m.
CONFIRMADA
MEMBRESIA
Socio 01
FIT-C001
Yoga
CLASE-04
12 nov 2026, 4:00 p.m.
Asistió
MEMBRESIA
© 2026 FitZone · Sistema integral de gestión · Equipo 11

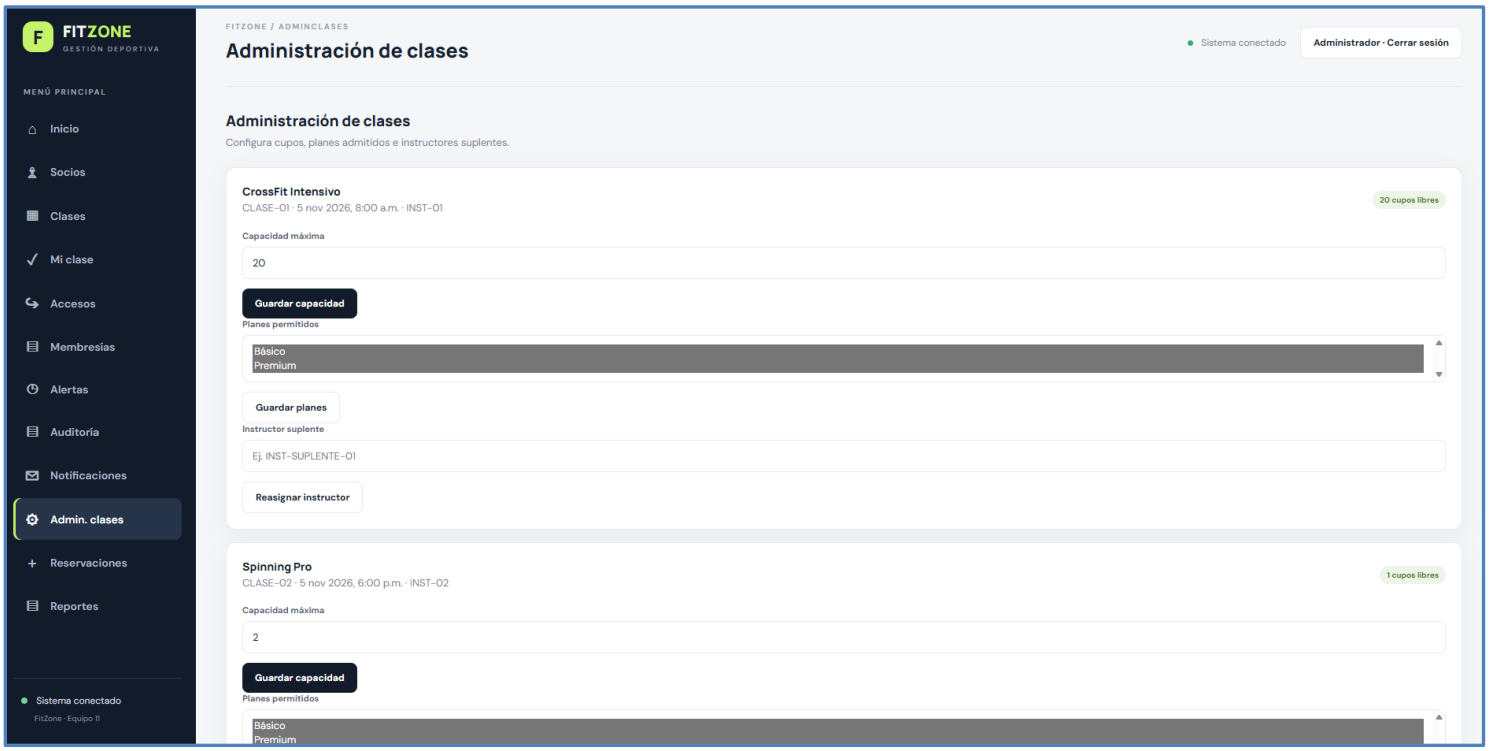

FITZONE
GESTIÓN DEPORTIVA
MENÚ PRINCIPAL
Inicio
Socios
Clases
Mi clase
Accesos
Membresías
Alertas
Auditoría
Notificaciones
Admin. clases
Reservaciones
Reportes
Sistema conectado
FitZone · Equipo 11
FITZONE / REPORTES
Reportes y analítica
Sistema conectado
Recepción · Cerrar sesión
Indicadores y desempeño
Información calculada a partir de la actividad registrada.
Actualizar reportes
Ocupación por clase
CrossFit Intensivo 0%
Spinning Pro 50%
Yoga Restaurativo 13%
Yoga 7%
Inasistencias registradas
SOCIO
NO ASISTIÓ
ASISTENCIAS
TASA HISTÓRICA
ESTADO
Socio 01 0 1 0.00% Regular
Socio 02 0 0 0.00% Regular
Socio 03 0 0 0.00% Regular
Socio 04 0 0 0.00% Regular
Socio 05 1 0 100.00% Regular
Socio 06 0 0 0.00% Regular
Socio 07 0 0 0.00% Regular
Socio 08 0 0 0.00% Regular
Socio 09 0 0 0.00% Regular
Socio 10 0 0 0.00% Regular
Socio 11 0 0 0.00% Regular
Socio 12 0 0 0.00% Regular
Socio 13 0 0 0.00% Regular

Admin (Solo Admin)

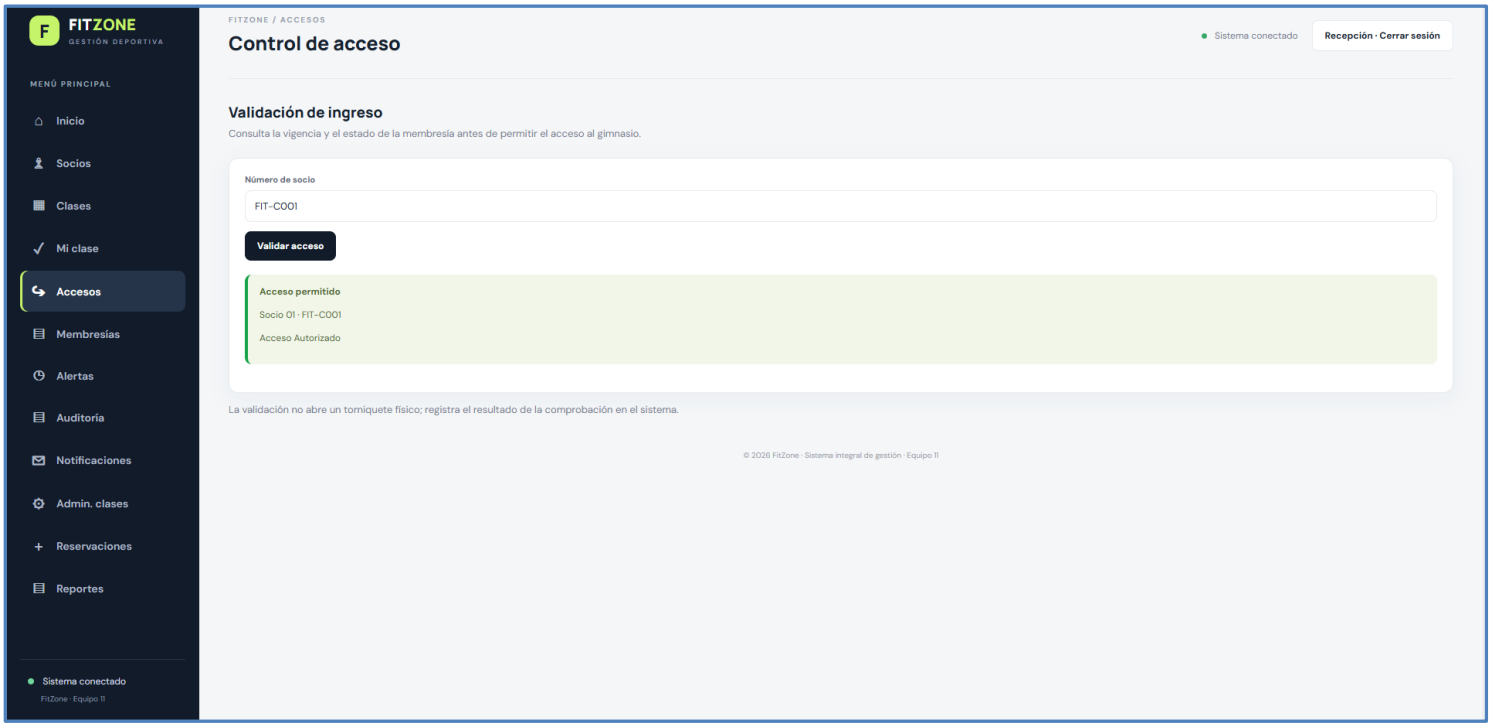




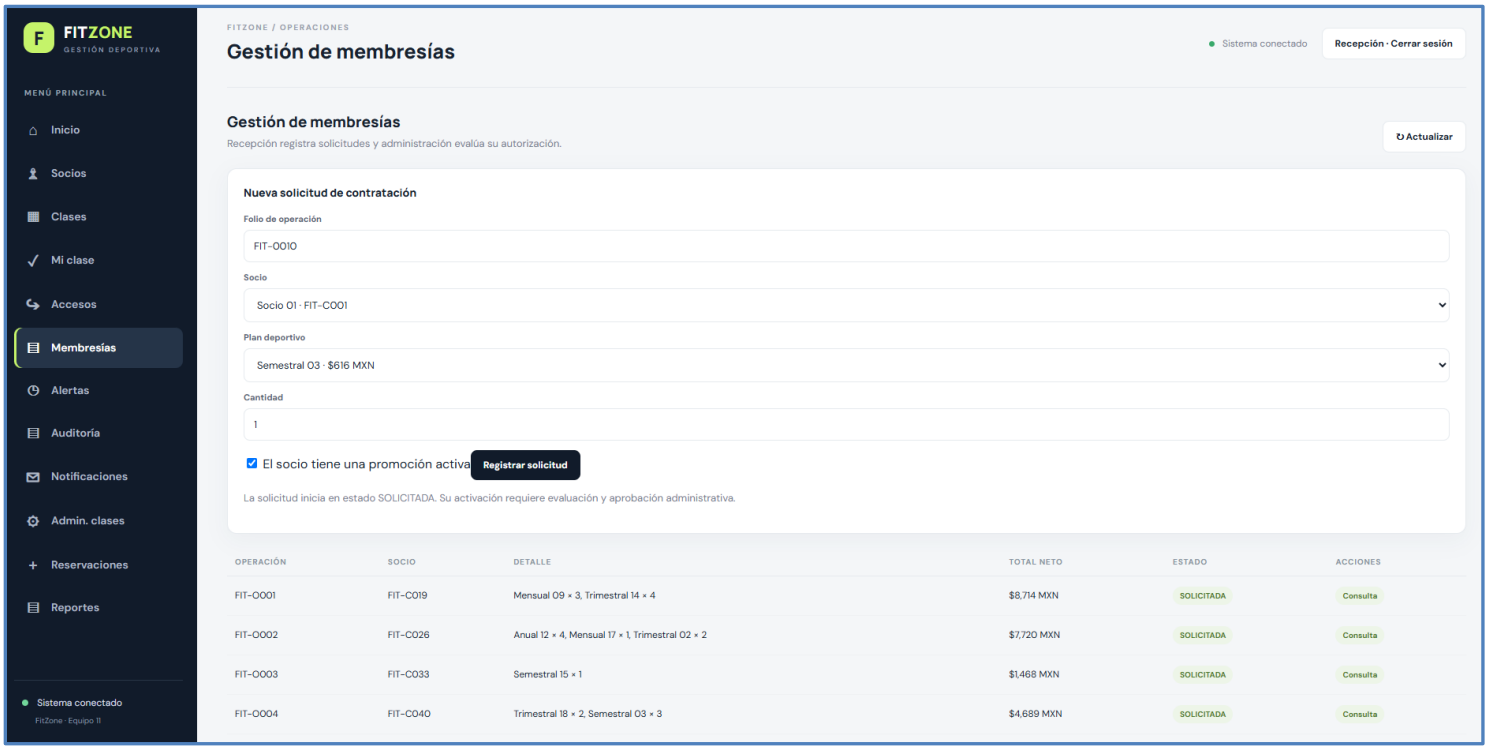

FITZONE
GESTIÓN DEPORTIVA
MENÚ PRINCIPAL
Inicio
Socios
Clases
Mi clase
Accesos
Membresías
Alertas
Auditoría
Notificaciones
Admin. clases
Reservaciones
Reportes
Sistema conectado
FitZone · Equipo 11
FITZONE / NOTIFICACIONES
Bandeja de notificaciones
Sistema conectado
Administrador · Cerrar sesión
Bandeja de notificaciones
Avisos generados por las operaciones del gimnasio.
Actualizar
Los mensajes se conservan en la cola interna (outbox). Esta pantalla permite consultarlos; el envío externo por correo o SMS aún requiere integrar un proveedor.
Avisos pendientes
7
Mensajes por entregar
FECHA
DESTINATARIO
ASUNTO
MENSAJE
ESTADO
5 nov 2026, 2:21 p.m.
FIT-C001
Estado de Reserva: CONFIRMADA
Su solicitud en la clase Spinning Pro quedó en estado CONFIRMADA.
Pendiente
5 nov 2026, 2:27 p.m.
FIT-C005
Estado de Reserva: CONFIRMADA
Su solicitud en la clase Yoga Restaurativo quedó en estado CONFIRMADA.
Pendiente
5 nov 2026, 2:32 p.m.
FIT-C001
Estado de Reserva: CONFIRMADA
Su solicitud en la clase Yoga Restaurativo quedó en estado CONFIRMADA.
Pendiente
5 nov 2026, 2:32 p.m.
FIT-C001
Estado de Reserva: CONFIRMADA
Su solicitud en la clase Yoga quedó en estado CONFIRMADA.
Pendiente
5 nov 2026, 2:33 p.m.
FIT-C005
Estado de Reserva: CONFIRMADA
Su solicitud en la clase CrossFit Intensivo quedó en estado CONFIRMADA.
Pendiente
6 nov 2026, 2:35 a.m.
FIT-C003
Lugar liberado
Tienes 30 minutos para confirmar la clase Spinning Pro.
Pendiente
6 nov 2026, 2:35 a.m.
FIT-C003
Lugar liberado
Tienes 30 minutos para confirmar la clase Spinning Pro.
Pendiente
© 2026 FitZone · Sistema integral de gestión · Equipo 11


FITZONE
GESTIÓN DEPORTIVA
MENÚ PRINCIPAL
Inicio
Socios
Clases
Mi clase
Accesos
Membresías
Alertas
Auditoría
Notificaciones
Admin. clases
Reservaciones
Reportes
Sistema conectado
FitZone · Equipo 11
FITZONE / ADMINCLASES
Administración de clases
Sistema conectado
Administrador · Cerrar sesión
Administración de clases
Configura cupos, planes admitidos e instructores suplentes.
CrossFit Intensivo
CLASE-01 · 5 nov 2026, 8:00 a.m. · INST-01
20 cupos libres
Capacidad máxima
20
Guardar capacidad
Planes permitidos
Básico
Premium
Guardar planes
Instructor suplente
Ej. INST-SUPLENTE-01
Reasignar instructor
Spinning Pro
CLASE-02 · 5 nov 2026, 6:00 p.m. · INST-02
1 cupos libres
Capacidad máxima
2
Guardar capacidad
Planes permitidos
Básico
Premium

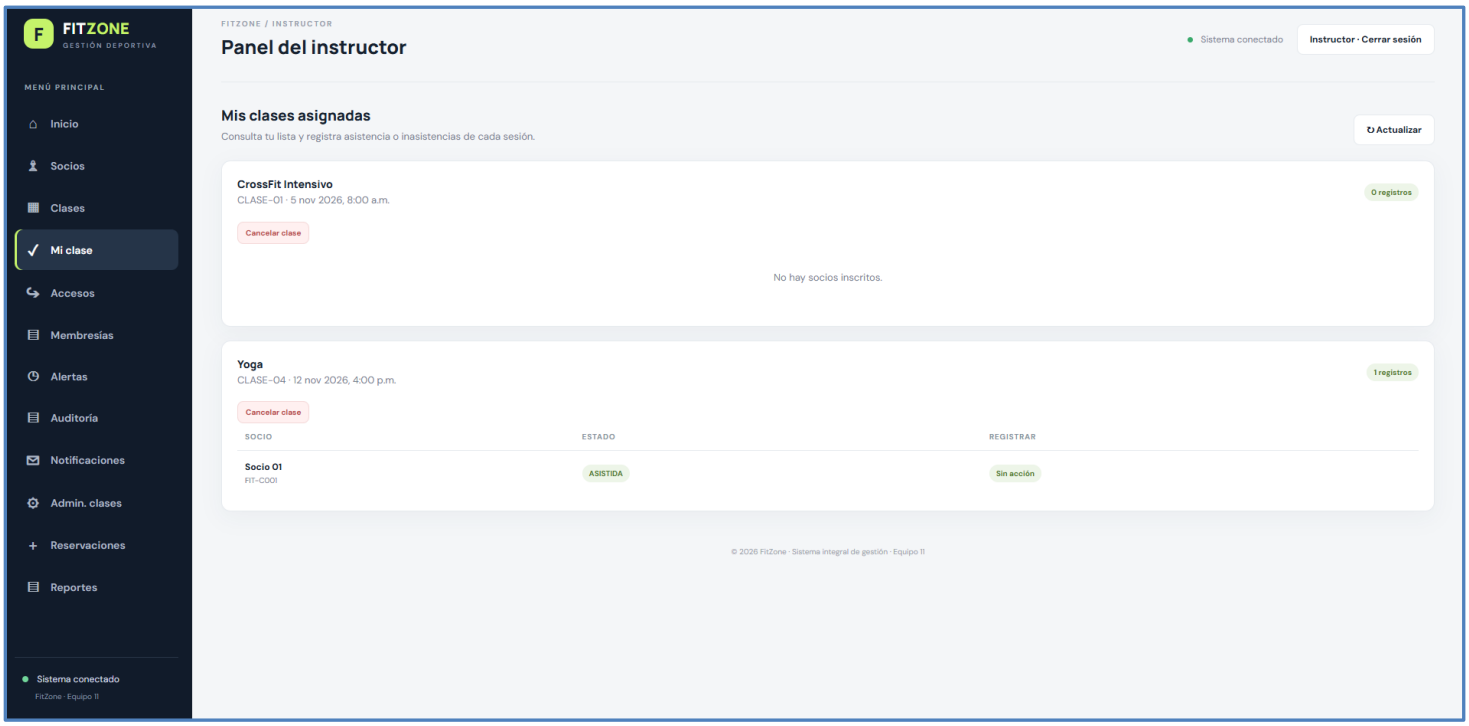
Recepción (Solo Recepción)



Admin y Recepción

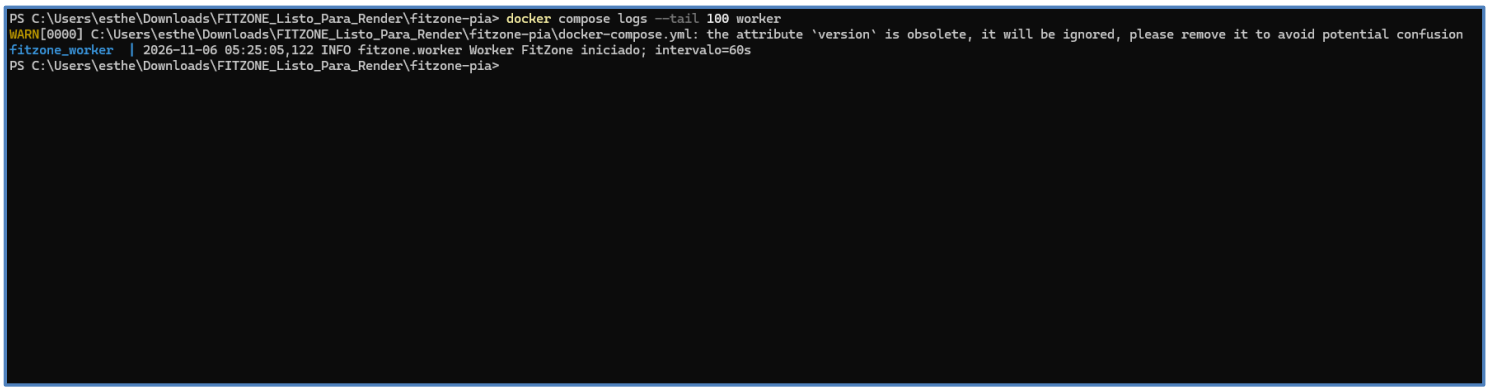


Instructor (Solo Instructor)



3.4 Worker y tareas programadas

El worker se ejecuta como un servicio separado. Revisa periódicamente situaciones como membresías vencidas y promociones de la lista de espera, y genera registros de notificación en una outbox. También contempla un resumen diario de alertas de vencimiento. La outbox representa persistencia interna de notificaciones: no equivale a un envío real de correo electrónico o SMS.



3.5 Vibe Coding e inteligencia artificial como apoyo

Durante el desarrollo se utilizó un enfoque de Vibe Coding asistido por inteligencia artificial: el equipo describió objetivos y restricciones en lenguaje natural, solicitó apoyo para proponer estructuras o fragmentos de código, revisó las respuestas y las integró de forma iterativa. La IA funcionó como herramienta de apoyo y no como sustituto de la comprensión técnica ni de la validación del equipo.

1. Análisis y descomposición: convertir requerimientos y reglas en tareas concretas, identificar dependencias y revisar consistencia entre documentos.
2. Diseño: explorar alternativas de organización por capas, contratos, modelos de dominio y patrones.
3. Implementación iterativa: generar o ajustar fragmentos de código, endpoints, pruebas y configuración a partir de errores observados.
4. Depuración: interpretar mensajes de error y proponer correcciones que después se comprobaron en el entorno real.
5. Validación humana: revisar que los cambios correspondieran al caso FitZone, ejecutar pruebas, inspeccionar resultados y evitar declarar como implementadas funciones que no estaban conectadas.

El criterio aplicado fue que una sugerencia de IA solo se considera parte del producto cuando el equipo la revisa, la incorpora al repositorio y verifica su comportamiento. La responsabilidad de las decisiones, la integración y la evidencia permanece en el equipo.

4. Fase de calidad, integración y producción

4.1 Pruebas automatizadas y cobertura

La suite de pruebas cubre comportamiento del dominio, servicios y repositorios, endpoints de la API, escenarios de datos y tareas programadas. En la ejecución local final dentro del contenedor de pruebas se obtuvieron 34 pruebas aprobadas y 91.43% de cobertura. El resultado supera el umbral de 80% solicitado por la guía.

Indicador	Resultado validado
Pruebas automatizadas	34 aprobadas
Cobertura de código	91.43%
Cobertura mínima solicitada	80%
Resultado	Cumple el umbral de cobertura

La cobertura indica qué proporción del código fue ejecutada por las pruebas; no demuestra por sí sola que todos los requisitos estén completos. Por ello se complementa con la matriz de trazabilidad y con la demostración de los flujos.

4.2 Docker Compose y ejecución local

La ejecución local se verificó con Docker Desktop y Docker Compose. El archivo docker-compose.yml define servicios independientes para PostgreSQL (db), la API (app), el worker y el contenedor de pruebas (tests). La separación permite reproducir el entorno sin instalar manualmente todas las dependencias del proyecto.

Durante la validación se comprobó la configuración con docker compose config, se construyeron e iniciaron los servicios db, app y worker, se verificó el estado de los contenedores, se consultó el endpoint /health, se abrió Swagger local, se ejecutó la carga inicial de datos y se corrió el contenedor de pruebas.

Comando/validación	Propósito
docker compose config	Comprobar que la configuración Compose sea válida.
docker compose up --build -d db app worker	Construir e iniciar PostgreSQL, API y worker.
docker compose ps	Comprobar que los tres servicios estén activos y que la base de datos esté saludable.
Invoke-RestMethod http://localhost:8000/health	Verificar que la API responde.
http://localhost:8000/docs	Inspeccionar la documentación interactiva de la API.
docker compose run --rm tests	Ejecutar pruebas y medir cobertura dentro del contenedor.

```
                                    > docker compose ps
WARN[0000] C:\Users\esthe\fitzone-pia-equipo-11\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME             IMAGE                          COMMAND                  SERVICE   CREATED         STATUS                   PORTS
fitzone_api      fitzone-pia-equipo-11-app      "uvicorn src.api.mai…"   app       5 minutes ago   Up 3 minutes             0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
fitzone_db       postgres:15-alpine             "docker-entrypoint.s…"   db        5 minutes ago   Up 4 minutes (healthy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
fitzone_worker   fitzone-pia-equipo-11-worker   "python -m src.jobs.…"   worker    5 minutes ago   Up 4 minutes             8000/tcp
PS C:\Users\esthe\fitzone-pia-equipo-11>
```

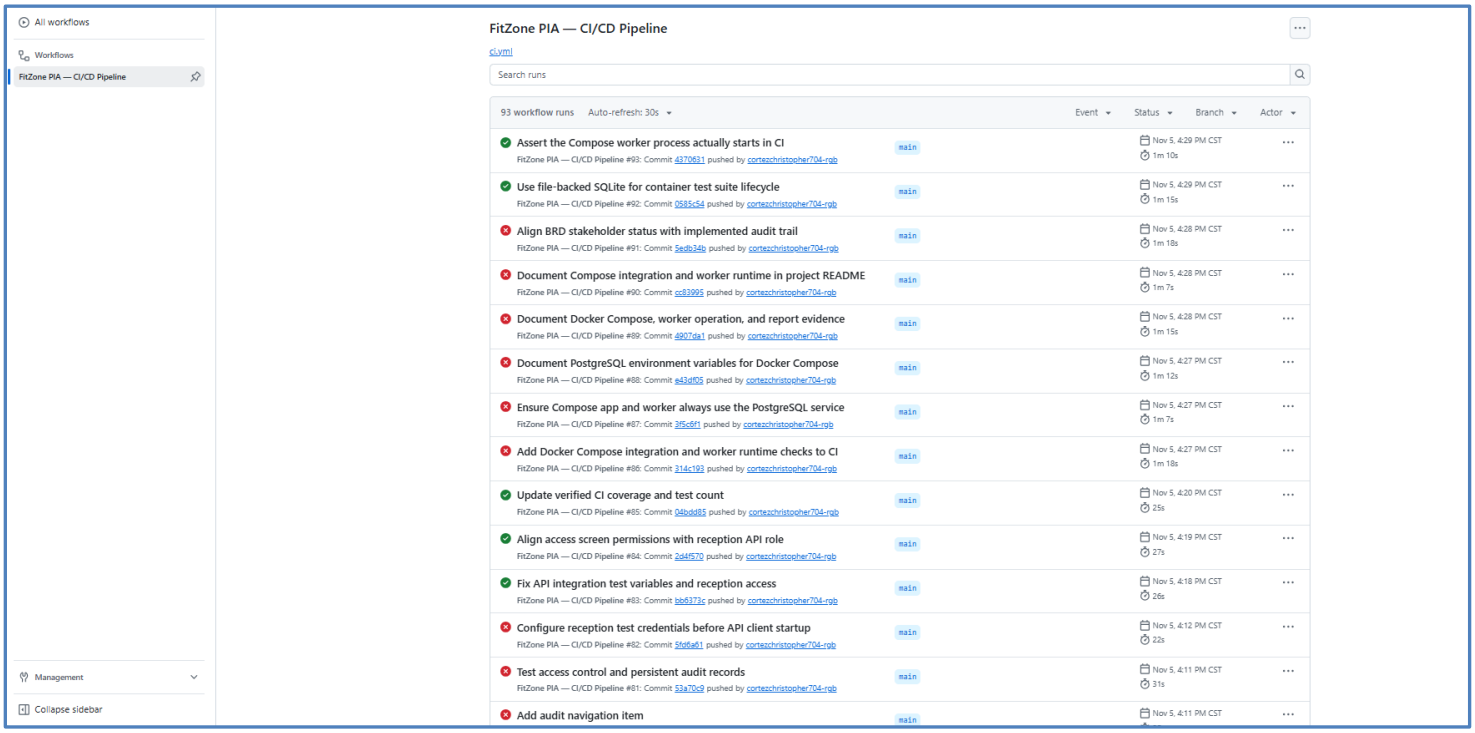
```
PS C:\Users\esthe\fitzone-pia-equipo-11> Invoke-RestMethod http://localhost:8000/health

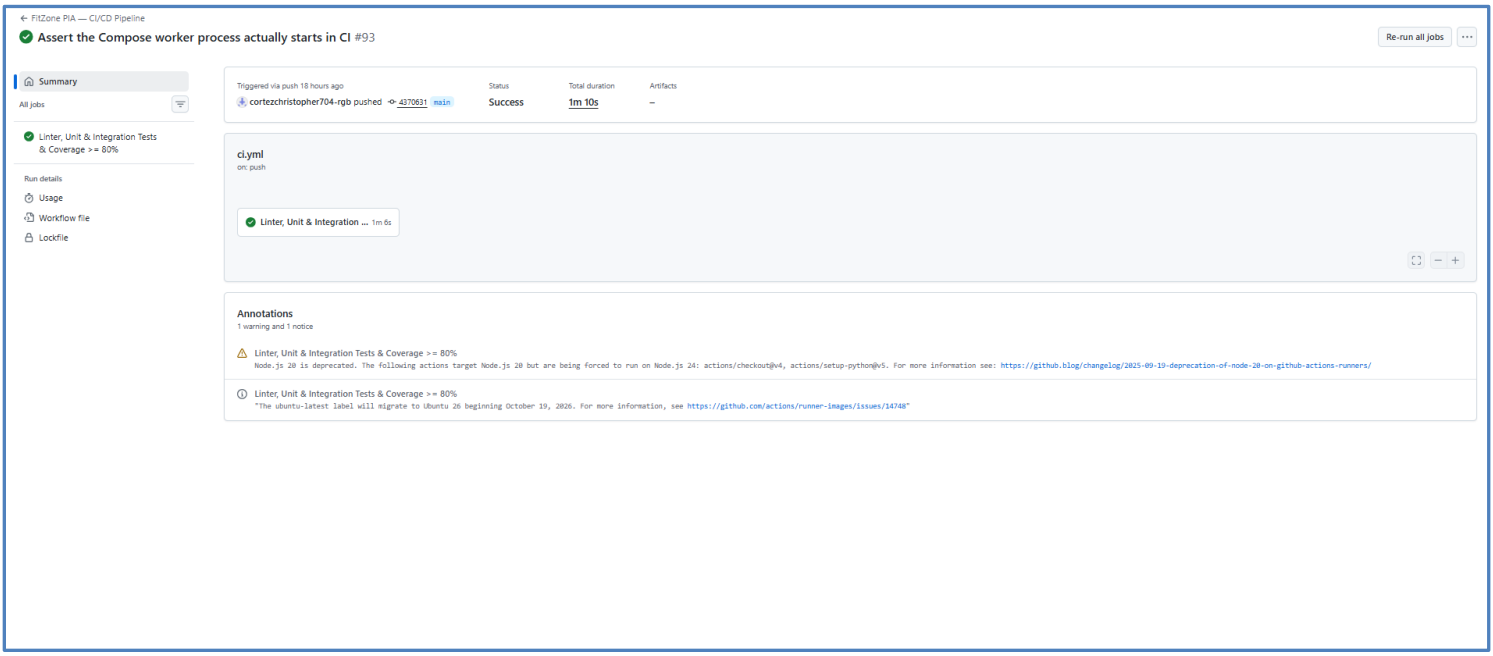
status service         equipo
------ -------         ------
ok     FitZone PIA API Equipo 11
```

```
PS C:\Users\esthe\fitzone-pia-equipo-11> docker compose run --rm tests
WARN[0000] C:\Users\esthe\fitzone-pia-equipo-11\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] run 1/1
 ✔Container fitzone_db Running                                                
```

```
plugins: anyio-4.16.0, cov-7.1.0
collected 34 items

tests/test_api.py::test_api_end_to_end_flows_and_status_codes PASSED                                                
```


All workflows
Workflows
FitZone PIA — CI/CD Pipeline
Management
Collapse sidebar
FitZone PIA — CI/CD Pipeline
ci.yml
Search runs
93 workflow runs
Auto-refresh: 30s
Event
Status
Branch
Actor
Assert the Compose worker process actually starts in CI
Use file-backed SQLite for container test suite lifecycle
Align BRD stakeholder status with implemented audit trail
Document Compose integration and worker runtime in project README
Document Docker Compose, worker operation, and report evidence
Document PostgreSQL environment variables for Docker Compose
Ensure Compose app and worker always use the PostgreSQL service
Add Docker Compose integration and worker runtime checks to CI
Update verified CI coverage and test count
Align access screen permissions with reception API role
Fix API integration test variables and reception access
Configure reception test credentials before API client startup
Test access control and persistent audit records
Add audit navigation item
main


FitZone PIA — CI/CD Pipeline
Assert the Compose worker process actually starts in CI #93
Re-run all jobs
Summary
All jobs
Linter, Unit & Integration Tests & Coverage >= 80%
Run details
Usage
Workflow file
Lockfile
Triggered via push 18 hours ago
cortezchristopher704-rgb pushed 4370631 main
Status
Success
Total duration
1m 10s
Artifacts
ci.yml
on: push
Linter, Unit & Integration ... 1m 6s
Annotations
1 warning and 1 notice
Linter, Unit & Integration Tests & Coverage >= 80%
Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/setup-python@v5. For more information see: https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/
Linter, Unit & Integration Tests & Coverage >= 80%
"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see https://github.com/actions/runner-images/issues/14748"

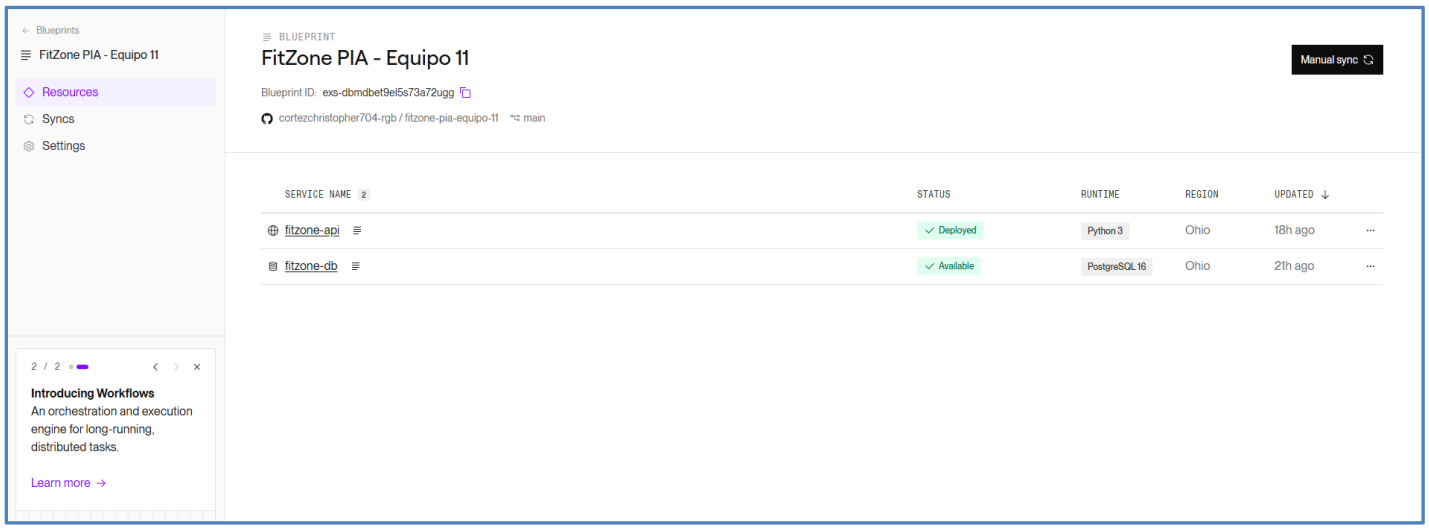
4.4 Despliegue y documentación de API

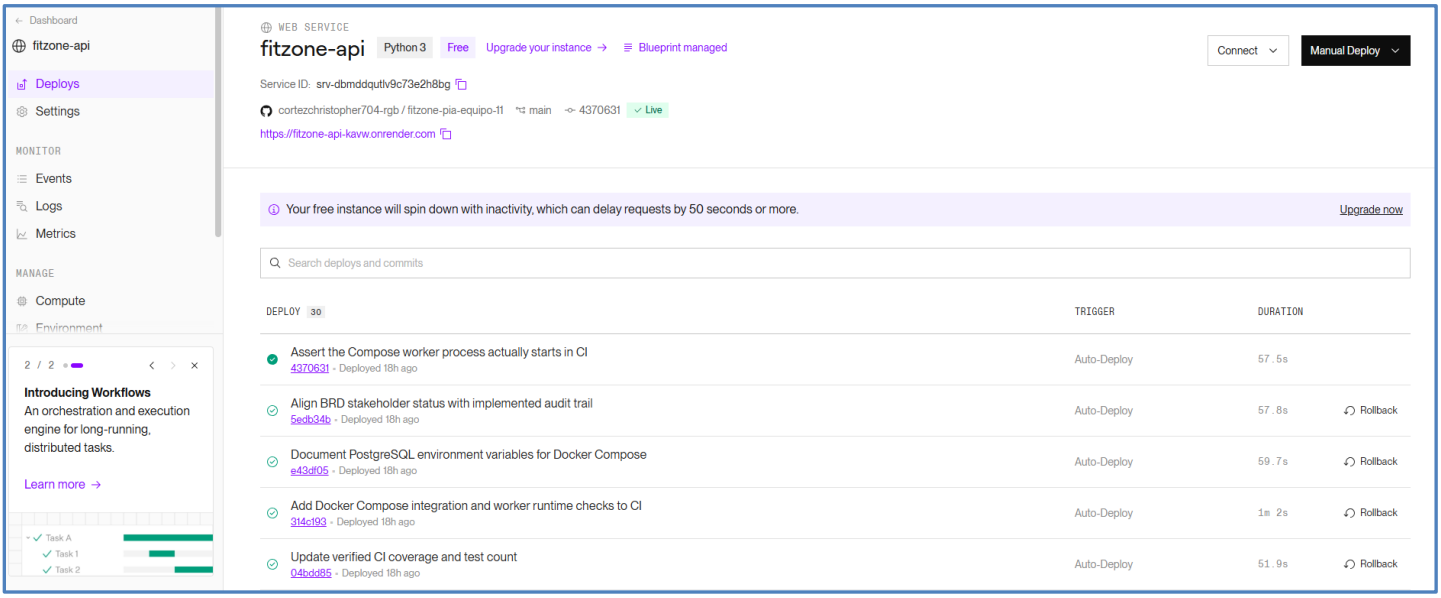
El servicio desplegado en Render expone la aplicación y la documentación Swagger/OpenAPI. Los enlaces de demostración son:

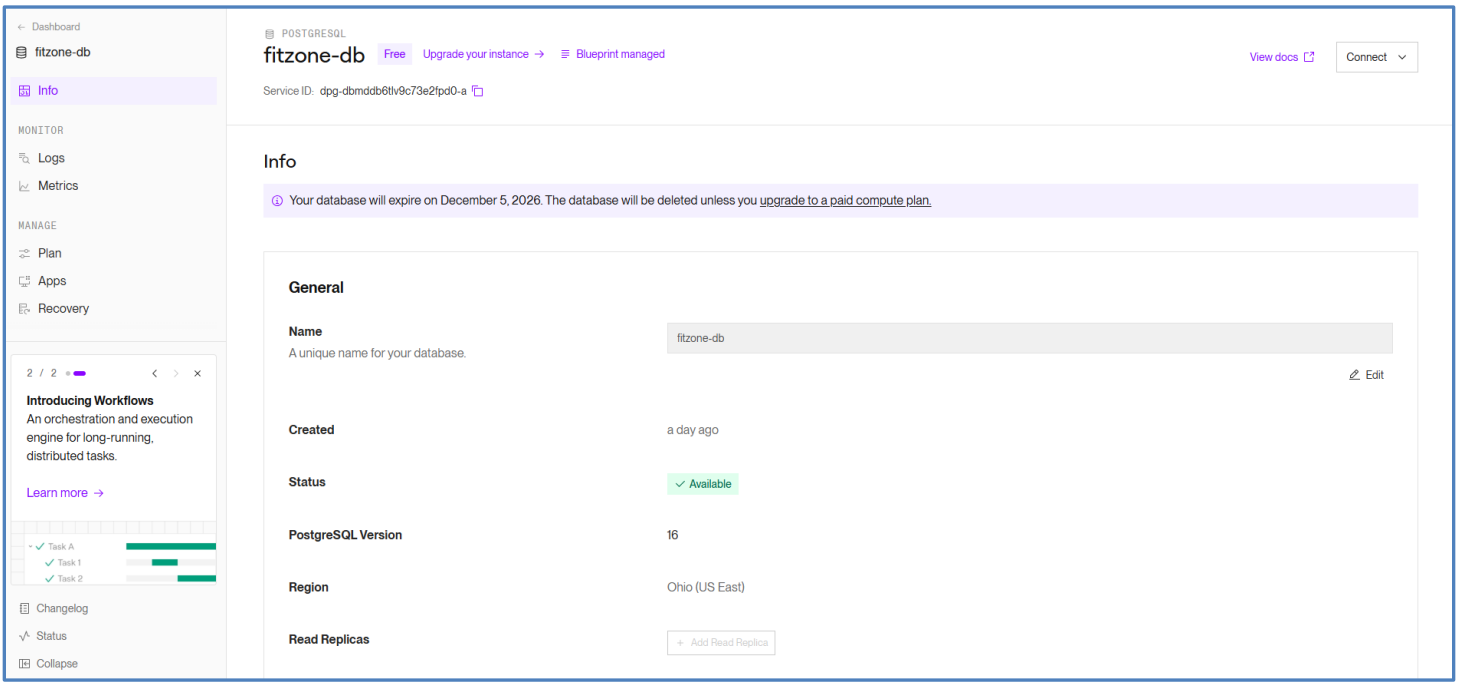
- Link Repositorio: https://github.com/cortezchristopher704-rgb/fitzone-pia-equipo-11
- Aplicación: https://fitzone-api-kavw.onrender.com/
- Swagger/OpenAPI: https://fitzone-api-kavw.onrender.com/docs
- Health check: https://fitzone-api-kavw.onrender.com/health

El worker se validó como servicio separado en Docker Compose local y en CI. No se afirma que el worker esté ejecutándose como proceso de fondo en el servicio gratuito de Render. De igual forma, el despliegue web no implica que existan integraciones físicas o proveedores externos que no fueron configurados.

Proyecto en Render:

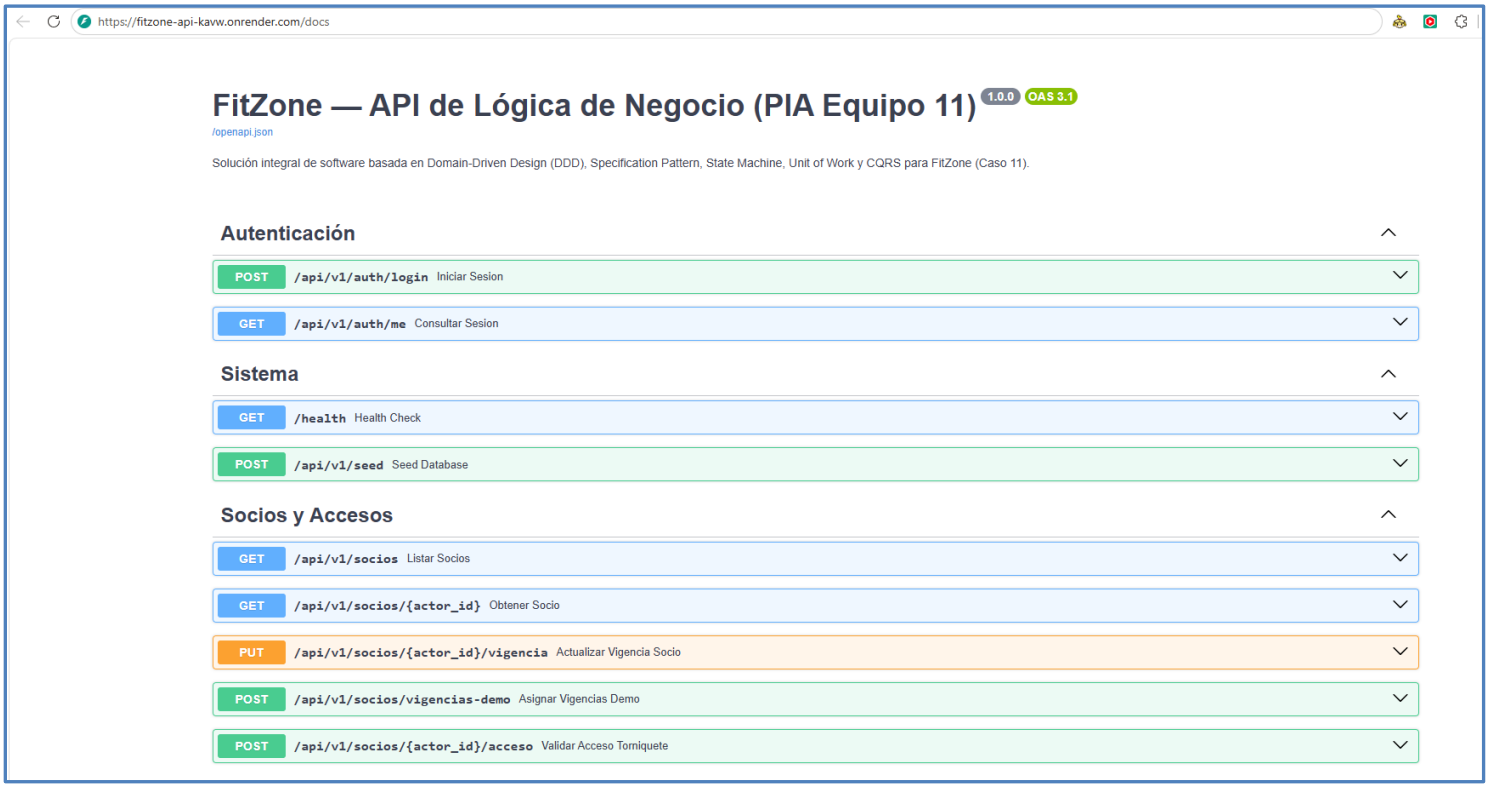




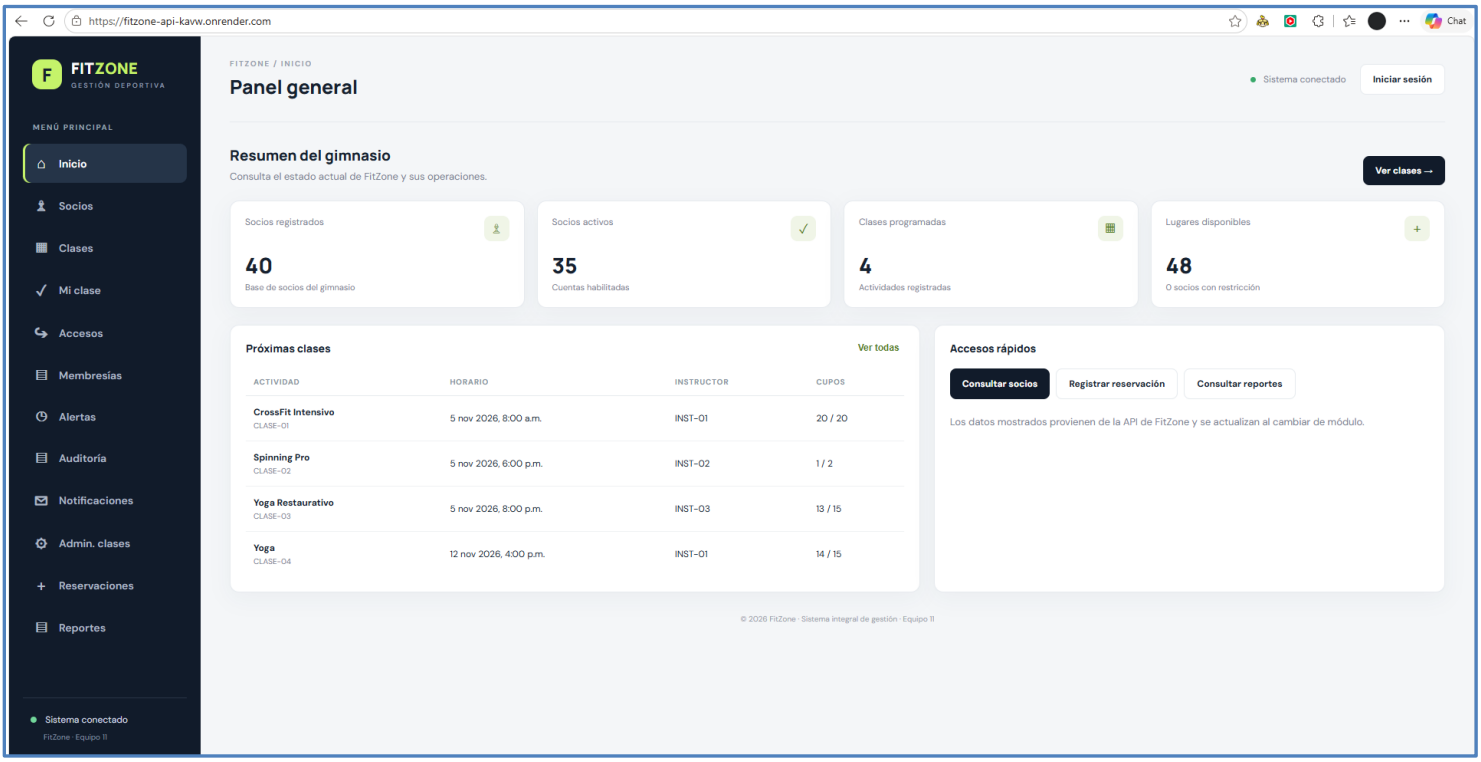


API en Render:

https://fitzone-api-kavw.onrender.com/docs



Aplicación en Render:
https://fitzone-api-kavw.onrender.com/



4.5 Seguridad de configuración y limitaciones

La configuración se separa del código mediante variables de entorno y el repositorio incluye .env.example como plantilla. Los secretos reales del archivo .env no deben incorporarse al repositorio ni mostrarse en capturas del reporte.

- No se integró un torniquete físico; el sistema contempla el puerto/contrato para esa integración, pero no controla un dispositivo real.
- No existe una pasarela de pago externa conectada; los flujos que requieren dicho proveedor no deben presentarse como cobros reales.
- Las notificaciones registradas en la outbox no se envían automáticamente por correo o SMS.
- El worker local está diseñado para una instancia; para escalar a varias instancias se requeriría coordinación distribuida o bloqueo.
- Unit of Work permite delimitar operaciones y rollback, pero no demuestra por sí solo aislamiento serializable ni elimina todas las condiciones de carrera ante concurrencia.
- Algunas historias de usuario permanecen parciales o no implementadas completamente; su estado debe consultarse en la matriz de trazabilidad.

5. Trazabilidad entre análisis, arquitectura, código y pruebas

La trazabilidad permite verificar que el sistema implementado responde al caso de negocio y no es una colección de componentes genéricos. La matriz del proyecto relaciona historias de usuario y reglas con elementos de dominio, servicios, endpoints y pruebas. La revisión de esta matriz también identifica funcionalidades parciales y brechas.

Artefacto de origen	Elemento técnico relacionado	Evidencia de verificación
BRD y reglas de negocio	Entidades, Value Objects, Specification y servicios.	Pruebas de dominio y servicios.
Historias Gherkin	Casos de uso y operaciones expuestas por API.	Pruebas unitarias/de integración y escenarios.
BPMN	Secuencia de servicios y transiciones del proceso.	Pruebas de flujos y máquina de estados.
Modelo DDD/UML	Agregados, entidades, puertos y adaptadores.	Código fuente y pruebas de repositorios.
Requisitos de calidad	Suite, workflow CI y Docker Compose.	34 pruebas, 91.43%, CI exitoso y ejecución local.

La matriz de trazabilidad de docs/phase2/MATRIZ_TRAZABILIDAD_FITZONE.md y su versión XLSX debe consultarse junto con este reporte. El estado parcial de una historia no se oculta: el reporte describe las capacidades demostradas y señala las extensiones pendientes.

Matriz de trazabilidad FitZone

Historia	Requisito	Estado	Código / ruta	Prueba existente	Brecha / nota
US-001	Reserva con vigencia, plan y cupo	Cumple	ClasePrograma da.reservar(); ReservaAppService.reservar_clase()	test_clase_programada_reserva_fifo_cutoff_and_drop_in; test_reservation_rejects...	Exige socio activo y vigencia registrada/vigente; el control de clases por plan se limita a planes incluidos configurados por clase, sin catálogo comercial completo.
US-002	Consultar disponibilidad y cobertura del plan	Cumple	GET /api/v1/clases; ClasePrograma da.cupos_disponibles	test_api_end_to_end_flows_and_status_codes	La consulta muestra disponibilidad; inclusión en plan no se calcula para todos los planes.
US-003	Lista de espera con posición FIFO	Cumple en dominio	ClasePrograma da.reservar(); lista_espera.append()	test_clase_programada_reserva_fifo_cutoff_and_drop_in	No se acredita consulta específica de posición mediante endpoint separado.
US-004	Cancelar con ≥3 h sin penalización y liberar cupo	Cumple	ClasePrograma da.cancelar_reserva(); ReservaAppService.cancelar_reserva()	test_clase_programada_reserva_fifo_cutoff_and_drop_in	La API calcula anticipación desde el horario persistido; los campos legacy del cliente se ignoran. Faltan notificaciones externas y horarios completos para registros heredados.

US-005	Promoción FIFO y ventana de confirmación 30 min	Cumple	confirmar_promocion(); expirar_promocion(); POST .../promociones/procesar-expiracion	test_waitlist_promotion_requires_confirmation_and_expires_fifo	Worker procesa la expiración cada 60 s y mantiene FIFO; falta notificación real al socio promovido.
US-006	Verificar vigencia al ingresar	Cumple	Socio.verificar_acceso_fisico(); PUT /api/v1/socios/{id}/vigencia; POST /api/v1/socios/{id}/acceso	test_api_end_to_end_flows_and_status_codes; test_socio_plan_basico_limit_no_shows_and_physical_access	Ahora deniega el acceso si no hay vigencia registrada y permite al administrador registrar fechas; falta integración con torniquete físico. El CSV oficial no contiene fechas contractuales.
US-007	Mostrar motivo de rechazo	Cumple en dominio/API	verificar_acceso_fisico(); POST /api/v1/socios/{id}/acceso	test_api_end_to_end_flows_and_status_codes	Motivos devueltos por la lógica; interfaz de recepción no incluida.
US-008	Recepción crea reserva manual	Cumple	POST /api/v1/clases/{id}/reservas/recepcion	test_new_use_cases_reception_instructor_capacity_and_expiry_alerts	Valida token/rol RECEPCION y empleado autenticado; falta directorio de usuarios y bitácora persistente.
US-009	Instructor consulta padrón de inscritos	Cumple	GET /api/v1/clases/{id}/instructor/{instructor_id}/lista	test_new_use_cases_reception_instructor_capacity_and_expiry_alerts	Incluye confirmados, asistidos y socios en lista de espera; valida token/rol e instructor asignado. Falta directorio de usuarios y bitácora persistente.
US-010	Instructor cancela ≥2 h, notifica y ofrece reagendar	Cumple	POST .../cancelar-instructor; SqlOutboxNotificador; notificaciones_outbox	test_new_use_cases_reception_instructor_capacity_and_expiry_alerts	Encola aviso persistente a cada socio afectado y administración; devuelve clases de la misma disciplina con cupo. Falta entrega externa y confirmación de reagendamiento.
US-011	Registrar asistencia / no-show	Cumple	AsistenciaAppService; POST .../asistencia y .../no-show	test_attendance_updates_reservation_state_without_erasing_monthly_no_shows; test_socio...	La regla de dominio existe; validar que API y persistencia actualicen la misma reserva/socio en todos los escenarios.
US-012	Administrador configura capacidad	Cumple	PATCH /api/v1/clases/{id}/capacidad; ClaseProgramada.actualizar_capacidad()	test_actualizar_capacidad_rechaza_aforo_menor_a_reservas_activas; test_new_use_cases_reception_instructor_capacity_and_expiry_alerts	Valida token/rol ADMIN, modifica aforo y protege reservas activas; falta directorio de usuarios y auditoría persistente.
US-013	Configurar clases incluidas en planes	Cumple	PATCH /api/v1/clases/{id}/planes-incluidos; ClaseProgramada.actualizar_planes_incluidos()	test_new_use_cases_reception_instructor_capacity_and_expiry_alerts	Configura y persiste tipos BASICO/PREMIUM por clase; falta catálogo de productos de plan y panel administrativo.
US-014	Reporte de ocupación	Cumple en consulta	ReportesQueryService.reporte_ocupacion_clases(); GET /api/v1/reportes/ocupacion	test_api_end_to_end_flows_and_status_codes	Reporte agregado disponible; validar filtros por periodo antes de afirmar analítica histórica.
US-015	Reporte de no-shows por socio	Cumple	ReportesQueryService.reporte_no_shows_socios(); GET /api/v1/reportes/no-shows	test_api_end_to_end_flows_and_status_codes	Calcula tasa porcentual histórica a partir de reservas con resultado ASISTIDA o NO_SHOW y conserva el contador mensual; falta filtro de periodo seleccionable y retención histórica fuera de las reservas actuales.
US-016	Alertas de membresías próximas a vencer	Cumple	POST /api/v1/reportes/alertas-vencimiento	test_new_use_cases_reception_instructor_capacity_and_expiry_alerts	Worker genera reporte diario estructurado en logs; falta canal de notificación al administrador.

US-017	3 no-shows/mes → bloqueo de reservas 7 días	Cumple en dominio	Socio.registrar_no_show(); actualizar_bloqueo(); ClaseProgramada.registrar_no_show()	test_socio_plan_basico_limit_no_shows_and_physical_access; test_monthly_no_show_counter_resets_by_month_and_block_expires	El contador y bloqueo están en dominio; el envío real de notificación depende de NotificadorPort.
US-018	Vencimiento cancela reservas y libera cupos	Cumple	procesar_membresias_vencidas(); worker de tareas programadas	test_worker_expires_membership_cancels_reservation_and_promotes_fifo; test_membership_expiry_transitions_active_contract_to_expired	El worker desactiva al socio vencido, vence operaciones activas/suspendidas, cancela reservas confirmadas y promociones pendientes y libera cupos; notificación externa aún no integrada.
US-019	Drop-in con pago único	Cumple	ServicioDeInscripcionExtraordinaria; PasarelaPagoPort; ReservaAppService	test_clase_programada_reserva_fifo_cutoff_and_drop_in	Sin pasarela configurada el pago falla de forma segura; falta proveedor real y prueba de cobro integrado.
US-020	Asignar instructor respaldo o cancelar	Cumple	POST .../reasignar-instructor; SqlOutboxNotificador; notificaciones_outbox	test_new_use_cases_reception_instructor_capacity_and_expiry_alerts	Rechaza suplente ya asignado a otra clase con el mismo inicio y encola avisos persistentes. No se puede validar solapamiento de duración porque el BRD no define duración oficial de clases; falta directorio/calendario completo y entrega externa.

6. Resultados obtenidos

- Se integraron los artefactos de análisis del caso FitZone y se mantuvo un conjunto de 20 historias de usuario con criterios Gherkin.
- Se documentó una arquitectura basada en DDD, UML, separación por capas, patrones, principios SOLID y ADRs.
- Se implementaron el dominio, los servicios de aplicación, la persistencia, la API REST, la interfaz web y un worker local.
- Se ejecutaron 34 pruebas automatizadas con 91.43% de cobertura.
- Se verificó Docker Compose local con PostgreSQL, API y worker activos, así como la respuesta del health check y Swagger.
- Se dispone de un workflow CI exitoso y de un servicio desplegado con Swagger/OpenAPI disponible.

7. Conclusiones individuales

Christopher Cortez Garcia — 2159313

El desarrollo del PIA de FitZone me permitió comprender cómo se conectan las distintas etapas de un proyecto de software, desde el análisis de las necesidades del negocio hasta la implementación y validación de una aplicación. Trabajar con DDD, Specification y State Machine ayudó a identificar la importancia de mantener las reglas dentro del dominio y no dispersarlas en distintas partes del sistema.

Además, comprobar el proyecto mediante Docker Compose, las pruebas automatizadas y la API permitió relacionar los conceptos vistos en clase con un entorno de ejecución real.

Oscar Elias Vera Moreno — 2160859

A lo largo del proyecto pude observar que las decisiones de arquitectura deben responder a problemas concretos del negocio. La separación de responsabilidades, el uso de repositorios y Unit of Work permiten organizar operaciones como las membresías y las reservas, además de facilitar las pruebas. También fue importante reconocer que una implementación funcional tiene límites: por ejemplo, la gestión de concurrencia, las integraciones externas y algunas historias parciales requieren trabajo adicional. Esto me permitió valorar tanto los resultados obtenidos como la necesidad de documentar honestamente lo que todavía falta.

Christofer Aaron Torres Puga — 2159639

La realización del PIA reforzó la importancia de mantener trazabilidad entre los requerimientos, el diseño, el código y las pruebas. Los diagramas y las historias de usuario sirven como guía, pero deben corresponder con lo que realmente se implementa. La organización por capas, los contratos de infraestructura y la automatización con GitHub Actions y Docker Compose ayudan a que el proyecto sea más fácil de revisar y reproducir. También comprendí que contar con una API documentada y pruebas automatizadas es fundamental para demostrar el funcionamiento de una solución.